



**TRIBHUVAN UNIVERSITY  
INSTITUTE OF ENGINEERING  
THAPATHALI CAMPUS**

**A REPORT  
ON  
FINE-TUNING LLAMA-2 7B FOR PYTHON CODE GENERATION**

**Submitted By:**

Deep Shrestha (35861)  
Pradip Pokhrel (35877)  
Rohan Dhakal (35882)  
Sanjay Shrestha (35888)

**Submitted To:**

Department of Electronics and Computer Engineering  
Thapathali Campus  
Kathmandu, Nepal

May, 2026



**TRIBHUVAN UNIVERSITY  
INSTITUTE OF ENGINEERING  
THAPATHALI CAMPUS**

**A REPORT  
ON  
FINE-TUNING LLAMA-2 7B FOR PYTHON CODE GENERATION**

**Submitted By:**

Deep Shrestha (35861)  
Pradip Pokhrel (35877)  
Rohan Dhakal (35882)  
Sanjay Shrestha (35888)

**Submitted To:**

Department of Electronics and Computer Engineering  
Thapathali Campus  
Kathmandu, Nepal

In partial fulfillment for the award of the  
Bachelors Degree in Computer Engineering

**Under the Supervision of**  
Asst. Prof. Suwarna Lingden

May, 2026

## DECLARATION

We hereby declare that the project entitled, “**Fine-tuning llama-2 7B for python code generation**” in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering **Bachelors in Computer Engineering** is a bona fide report of the work carried out by us. These materials contained within the report have not been submitted to any University or Institution for the award of any degree, and we are the only author of this complete work and no sources other than the listed ones have been used in this work.

Deep Shrestha

(THA079BCT011)

---

Pradip Pokhrel

(THA079BCT027)

---

Rohan Dhakal

(THA079BCT033)

---

Sanjay Shrestha

(THA079BCT039)

---

**Date:** May, 2026

### **CERTIFICATE OF APPROVAL**

The undersigned certify that they have read and recommended to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, a minor project work entitled “**Fine-tuning llama-2 7B for python code generation**” submitted by **Deep Shrestha, Pradip Pokhrel, Rohan Dhakal, Sanjay Shrestha** in partial fulfillment for the award of the degree of Bachelor of Engineering in Computer Engineering. The project was carried out under the supervision and within the time prescribed by the syllabus.

We found the students to be hardworking, skilled, and ready to undertake any work related to their field of study. Hence, we recommend the award of the degree of Bachelor of Engineering in Computer Engineering.

---

Project Supervisor

Asst. Prof. Suwarna Lingden

Department of Electronics and Computer  
Engineering

Thapathali Campus, IOE

---

External Examiner

Dr. Suman Raj Bista

Nepal Applied Mathematics and  
Informatics Institute for Research

Jwagal, Lalitpur

---

Head of Department

Er. Umesh Kanta Ghimire

Department of Electronics and Computer  
Engineering

Thapathali Campus, IOE

**Date:** May, 2026

## **COPYRIGHT**

The author has agreed that the library, Department of Electronics and Computer Engineering, Thapathali Campus, may make this report freely available for inspection. Moreover, the author has agreed that the permission for extensive copying of this project work for scholarly purpose may be granted by the professor/lecturer, who supervised the project work recorded herein or, in their absence, by the head of the department. It is understood that the recognition will be given to the author of this report and to the Department of Electronics and Computer Engineering, IOE, Thapathali Campus in any use of the material of this report. Copying of publication or other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, IOE, Thapathali Campus and author's written permission is prohibited.

Request for permission to copy or to make any use of the material in this project in whole or part should be addressed to Department of Electronics and Computer Engineering, IOE, Thapathali Campus.

## ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Institute of Engineering, Tribhuvan University, for incorporating a minor project into the curriculum of the Bachelor's in Computer Engineering program.

We extend our heartfelt thanks to our supervisor, **Asst. Prof. Suwarna Lingden**, for his continuous guidance, valuable feedback, and unwavering support throughout the project. His expertise and encouragement have played a crucial role in shaping this work.

We would also like to thank the Department of Electronics and Computer Engineering, Thapathali Campus, for their continuous assistance and recommendations during the project selection process, which significantly contributed to our professional development.

Finally, we are grateful to all the teachers, friends, and both direct and indirect contributors for their valuable insights, encouragement, and support throughout this journey.

|                 |                |
|-----------------|----------------|
| Deep Shrestha   | (THA079BCT011) |
| Pradip Pokhrel  | (THA079BCT027) |
| Rohan Dhakal    | (THA079BCT033) |
| Sanjay Shrestha | (THA079BCT039) |

**Date:** May, 2026

## ABSTRACT

Large Language Models (LLMs) have exhibited remarkable capabilities in natural language understanding and code generation. However, their deployment in resource-constrained environments remains limited due to substantial computational and memory requirements. This study proposes an efficient methodology for Python code generation by fine-tuning the LLaMA-2 7B chat model using the Quantized Low-Rank Adaptation (QLoRA) technique. A high-quality training dataset is curated and refined through a teacher-model-based filtering mechanism, while structured prompt design is employed to enhance logical consistency in generated outputs. Evaluation on the HumanEval benchmark shows that the fine-tuned model improves Pass@1 from 0.1159 to 0.1280 and Pass@5 from 0.1890 to 0.2256 relative to the base model. Code quality is assessed using BLEU and CodeBLEU metrics, yielding scores of 10.2703 and 0.2363 respectively. Local deployment via Ollama enables efficient inference, demonstrating the practical viability of the proposed approach. This work underscores the potential of PEFT techniques in optimizing LLMs for code generation tasks while maintaining performance, paving the way for broader accessibility and application in real-world scenarios.

**Keywords**—LLMs, LLAMA2, Code Generation, PEFT, QLoRA, LoRA, Hugging Face, Ollama, Local Inference, Prompt Engineering.

## TABLE OF CONTENTS

|                                                                |             |
|----------------------------------------------------------------|-------------|
| <b>DECLARATION . . . . .</b>                                   | <b>i</b>    |
| <b>CERTIFICATE OF APPROVAL . . . . .</b>                       | <b>ii</b>   |
| <b>COPYRIGHT . . . . .</b>                                     | <b>iii</b>  |
| <b>ACKNOWLEDGEMENT . . . . .</b>                               | <b>iv</b>   |
| <b>ABSTRACT . . . . .</b>                                      | <b>v</b>    |
| <b>LIST OF FIGURES . . . . .</b>                               | <b>viii</b> |
| <b>LIST OF TABLES . . . . .</b>                                | <b>ix</b>   |
| <b>LIST OF ABBREVIATIONS . . . . .</b>                         | <b>ix</b>   |
| <b>1 INTRODUCTION . . . . .</b>                                | <b>1</b>    |
| 1.1 Background . . . . .                                       | 1           |
| 1.2 Motivation . . . . .                                       | 1           |
| 1.3 Problem Definition . . . . .                               | 1           |
| 1.4 Objectives . . . . .                                       | 1           |
| 1.5 Scope . . . . .                                            | 2           |
| <b>2 LITERATURE REVIEW . . . . .</b>                           | <b>3</b>    |
| 2.1 Transformer Architecture and Attention Mechanism . . . . . | 3           |
| 2.2 Scaling Laws and Transfer Learning . . . . .               | 3           |
| 2.3 Few shot learning and in context learning . . . . .        | 3           |
| 2.4 LoRA . . . . .                                             | 3           |
| 2.5 QLoRA . . . . .                                            | 3           |
| 2.6 PEFT . . . . .                                             | 4           |
| 2.7 Open LLMs . . . . .                                        | 4           |
| 2.8 Domain Specific Application . . . . .                      | 4           |
| 2.9 Evaluation metrics for code generation . . . . .           | 4           |
| 2.10 Multilingual code generations benchmarks . . . . .        | 4           |
| 2.11 Training with noisy labels . . . . .                      | 5           |
| <b>3 REQUIREMENTS ANALYSIS . . . . .</b>                       | <b>6</b>    |
| 3.1 Hardware requirements . . . . .                            | 6           |
| 3.2 Software requirements . . . . .                            | 6           |
| 3.3 Feasibility Analysis . . . . .                             | 8           |



|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>4</b> | <b>SYSTEM ARCHITECTURE AND METHODOLOGY</b>         | <b>10</b> |
| 4.1      | System Architecture of LLaMA-2                     | 10        |
| 4.2      | Code Generation with LLMs                          | 23        |
| 4.3      | Dataset Collection and Preparation                 | 25        |
| 4.4      | Hyperparameter Tuning                              | 28        |
| 4.5      | Quantized Low-Rank Adaptation (QLoRA)              | 30        |
| 4.6      | Fine-Tuning                                        | 34        |
| 4.7      | Evaluation                                         | 36        |
| 4.8      | Ollama for Local Inference                         | 39        |
| <b>5</b> | <b>IMPLEMENTATION DETAILS</b>                      | <b>41</b> |
| 5.1      | Data Collection and Preparation                    | 41        |
| 5.2      | Hyperparameter Optimization                        | 47        |
| 5.3      | Fine-Tuning Implementation                         | 50        |
| 5.4      | Evaluation                                         | 53        |
| 5.5      | Ollama Implementation for Local Inference          | 56        |
| <b>6</b> | <b>RESULTS AND ANALYSIS</b>                        | <b>63</b> |
| 6.1      | Validation Loss During Fine Tuning                 | 63        |
| 6.2      | Effect of Quantization and Performance Assessment  | 64        |
| 6.3      | Hyperparameter Optimization and Training Stability | 66        |
| 6.4      | Evaluation on Quantitative Metrics                 | 68        |
| 6.5      | Functional Evaluation: Pass@k on HumanEval         | 70        |
| 6.6      | Functional Evaluation of CodeLlama                 | 71        |
| 6.7      | System Performance in Deployment (Ollama)          | 76        |
| 6.8      | Summary                                            | 77        |
| <b>7</b> | <b>FUTURE ENHANCEMENT</b>                          | <b>78</b> |
| 7.1      | Limitations                                        | 78        |
| 7.2      | Future Work                                        | 78        |
| <b>8</b> | <b>CONCLUSION</b>                                  | <b>81</b> |
| 8.1      | Key Achievements                                   | 81        |
| 8.2      | Final Remarks                                      | 81        |
| <b>9</b> | <b>APPENDICES</b>                                  | <b>83</b> |
|          | <b>REFERENCES</b>                                  | <b>90</b> |

## LIST OF FIGURES

|             |                                                                                                          |    |
|-------------|----------------------------------------------------------------------------------------------------------|----|
| Figure 4-1  | LLaMA-2 Architecture . . . . .                                                                           | 10 |
| Figure 4-2  | Tokenization Process . . . . .                                                                           | 17 |
| Figure 4-3  | Multi-Head Attention Mechanism . . . . .                                                                 | 19 |
| Figure 4-4  | Rotary Positional Embeddings . . . . .                                                                   | 20 |
| Figure 4-5  | Softmax Output Layer . . . . .                                                                           | 22 |
| Figure 4-6  | Llama Model with QLoRA . . . . .                                                                         | 31 |
| Figure 4-7  | Double Quantization . . . . .                                                                            | 32 |
| Figure 4-8  | Flow Diagram of the Fine-Tuning Process . . . . .                                                        | 34 |
| Figure 5-1  | Implementation of the Project . . . . .                                                                  | 41 |
| Figure 5-2  | Dataset Preparation Pipeline . . . . .                                                                   | 41 |
| Figure 5-3  | Alpaca Datasets . . . . .                                                                                | 43 |
| Figure 5-4  | FlyTech Datasets . . . . .                                                                               | 43 |
| Figure 5-5  | Final Datasets . . . . .                                                                                 | 47 |
| Figure 5-6  | Hyperparameter Optimization with Random Search . . . . .                                                 | 48 |
| Figure 5-7  | GPU Usage During the Fine-Tuning Process, Showing Utilization<br>Trends Across the Dual T4 GPUs. . . . . | 53 |
| Figure 5-8  | Ollama Inference . . . . .                                                                               | 57 |
| Figure 5-9  | Use Case Diagram . . . . .                                                                               | 58 |
| Figure 5-10 | Class Diagram . . . . .                                                                                  | 59 |
| Figure 5-11 | Sequence Diagram . . . . .                                                                               | 60 |
| Figure 5-12 | Component Diagram . . . . .                                                                              | 61 |
| Figure 5-13 | Activity Diagram . . . . .                                                                               | 62 |
| Figure 6-1  | Training and Validation Loss Trajectory. . . . .                                                         | 63 |
| Figure 6-2  | BLEU And CodeBLEU Score Of Quantized And Unquantized Model                                               | 65 |
| Figure 6-3  | Evaluation Loss In Different Trials During Hyperparameter Tuning .                                       | 67 |
| Figure 6-4  | BLEU And CodeBLEU Score Of Fine-Tuned And Quantized Model                                                | 68 |
| Figure 6-5  | Pass@k Score Of Quantized And Fine-Tuned Model . . . . .                                                 | 70 |
| Figure 6-6  | Pass@k Score Of Fine-Tuned And CodeLlama Model . . . . .                                                 | 72 |
| Figure 6-7  | Resource Usage By Ollama During Inference . . . . .                                                      | 77 |
| Figure 9-1  | Project Schedule (Gantt Chart) . . . . .                                                                 | 87 |

## LIST OF TABLES

|                                                                                        |    |
|----------------------------------------------------------------------------------------|----|
| Table 3-1 Infrastructure Allocation and Hardware Resources . . . . .                   | 6  |
| Table 4-1 Architectural Variants of LLaMA-2 Models . . . . .                           | 11 |
| Table 4-2 Context Length of LLaMA-2 Models . . . . .                                   | 11 |
| Table 4-3 Step-by-Step BPE Token Merges . . . . .                                      | 17 |
| Table 4-4 Comparison of Classification Engines for Data Preprocessing . . . . .        | 26 |
| Table 5-1 Datasets Used for Fine-tuning Llama-2 7B for Python Code Generation. . . . . | 42 |
| Table 5-2 Summary of Dataset Refinement Metrics . . . . .                              | 44 |
| Table 5-3 Impact of Repetition Ratio Threshold on Dataset Volume . . . . .             | 45 |
| Table 5-4 Data Filtering Heuristics and Constraints Summary . . . . .                  | 45 |
| Table 5-5 Dataset Distillation and Logic Density Improvement . . . . .                 | 45 |
| Table 5-6 Preprocessing Impact of Docstring Removal . . . . .                          | 46 |
| Table 5-7 Final Fine-Tuning Hyperparameters and Training Configuration . . . . .       | 52 |
| Table 6-1 Detailed Training and Validation Loss Logs . . . . .                         | 64 |
| Table 6-2 Performance Degradation Analysis: 16-bit vs. 4-bit Quantization . . . . .    | 65 |
| Table 6-3 Selected Optimal Hyperparameters (Trial 43) . . . . .                        | 67 |
| Table 6-4 Final Model Performance on Test Set ( $N = 1392$ ) . . . . .                 | 69 |
| Table 6-5 HumanEval Functional Correctness Results (Pass@k) . . . . .                  | 71 |
| Table 6-6 CodeLlama Functional Correctness Results (Pass@k) . . . . .                  | 72 |
| Table 9-1 Optuna Hyperparameter Optimization Results . . . . .                         | 84 |
| Table 9-2 Estimated Cost Breakdown . . . . .                                           | 86 |

## LIST OF ABBREVIATIONS

|         |                                              |
|---------|----------------------------------------------|
| AdamW   | Adam with Decoupled Weight Decay             |
| AI      | Artificial Intelligence                      |
| API     | Application Programming Interface            |
| AST     | Abstract Syntax Tree                         |
| BF16    | Brain Floating Point 16                      |
| BLEU    | Bilingual Evaluation Understudy              |
| BP      | Brevity Penalty                              |
| BPE     | Byte Pair Encoding                           |
| BS      | Batch Size                                   |
| CDF     | Cumulative Distribution Function             |
| CI/CD   | Continuous Integration/Continuous Deployment |
| CLI     | Command Line Interface                       |
| CoT     | Chain-of-Thought                             |
| CP      | Competitive Programming                      |
| CPU     | Central Processing Unit                      |
| DFScore | Data-Flow Match Score                        |
| DoRA    | Weight-Decomposed Low-Rank Adaptation        |
| DQ      | Double Quantization                          |
| DVC     | Data Version Control                         |
| EOS     | End-of-Sequence                              |
| FFN     | Feed-Forward Network                         |
| FLOPS   | Floating Point Operations per Second         |
| FP16    | Half-Precision Floating-Point                |
| FP8     | 8-bit Floating Point                         |
| GGUF    | GPT-Generated Unified Format                 |
| GLU     | Gated Linear Unit                            |

|         |                                            |
|---------|--------------------------------------------|
| GPT     | Generative Pre-trained Transformer         |
| GPU     | Graphics Processing Unit                   |
| GQA     | Grouped Query Attention                    |
| HPO     | Hyperparameter Optimization                |
| IDE     | Integrated Development Environment         |
| LLaMA   | Large Language Model Meta AI               |
| LLM     | Large Language Model                       |
| LoRA    | Low-Rank Adaptation                        |
| MHA     | Multi-Head Attention                       |
| MLP     | Multi-Layer Perceptron                     |
| NF4     | 4-bit NormalFloat                          |
| NLP     | Natural Language Processing                |
| OOM     | Out of Memory                              |
| OOV     | Out of Vocabulary                          |
| OpenAI  | Open Artificial Intelligence               |
| OPT     | Open Pre-trained Transformer               |
| PEFT    | Parameter-Efficient Fine-Tuning            |
| PII     | Personally Identifiable Information        |
| PLM     | Pre-trained Language Model                 |
| QLoRA   | Quantized Low-Rank Adaptation              |
| REST    | Representational State Transfer            |
| RLHF    | Reinforcement Learning with Human Feedback |
| RMSNorm | Root Mean Square Layer Normalization       |
| RoPE    | Rotary Positional Embedding                |
| RTX     | Ray Tracing Texel eXtreme                  |
| SFT     | Supervised Fine-Tuning                     |
| SOS     | Start of Sequence                          |

|        |                                                              |
|--------|--------------------------------------------------------------|
| SwiGLU | Swish-Gated Linear Unit                                      |
| T4     | NVIDIA T4 GPU                                                |
| TPE    | Tree-structured Parzen Estimator                             |
| TPU    | Tensor Processing Unit                                       |
| TRL    | Training Reinforcement Learning                              |
| UI     | User Interface                                               |
| ULMFiT | Universal Language Model Fine-tuning for Text Classification |
| UTF-8  | Unicode Transformation Format 8-bit                          |
| VRAM   | Video Random Access Memory                                   |
| WandB  | Weights and Biases                                           |

# **1 INTRODUCTION**

## **1.1 Background**

With the advent of Large Language Model (LLM) software development process has completely changed as they have become necessity for modern workflows, assisting programmers in tasks such as code generation, debugging, and documentation. Even with this success, there are studies like [1] which shows LLM requires huge datasets and computer resources for improved performance, and this dependence on Graphical Processing Unit (GPU) for training and inference makes them very difficult to use locally on limited resources.

## **1.2 Motivation**

Advance LLMs are mostly accessed through API for code generation, although smaller model like Llama-2 7B [2] could run on single consumer GPU, their performance is not up-to the mark due to limited parameter scale. Our initial research and experiment shown that python code generation from small models locally resulted in poor code quality and high error rate, this gap in small model could be filled with the help of process called fine-tuning (process which specializes general model for specific tasks) [3]. However, full fine-tuning requires extensive resources which can be addressed using Parameter-Efficient Fine-Tuning (PEFT) through Quantized Low-Rank Adaptation (QLoRA) [4], it quantizes model to 4-bit precision and trains only minimal set of adapter weights [5], achieving performance similar to full fine-tuning. So, our goal is to fine-tune Llama-2 7B using QLoRA on python dataset, which is prepared through LLM assisted processing and manual processing.

## **1.3 Problem Definition**

The Llama-2 7B generates very poor quality python code in its standard form, generated code is mostly syntactically incorrect or logically flawed compared to the proprietary systems [6]. Also, full fine-tuning is inaccessible due to high resources demand as the VRAM requirement far exceeds the consumer grade GPUs like RTX 3060. The model faces syntax, logical, semantic, dataflow issue in its standard form.

## **1.4 Objectives**

The primary objective of this project is:

- To apply QLoRA for efficient compression and fine tuning of Llama-2 7B model so that it can run on standard consumer hardware for python code generation.

## **1.5 Scope**

The focus of this project is to refine Llama-2 7B for Python programming, enhancing its ability to handle complex and various libraries. The training is kept within 14GB VRAM limit with the help of 4-bit QLoRA, making sure that the system remains compatible with consumer grade hardware and google colab. The dataset to train the model was gathered manually or sourced from standard sources such as alpaca and flytech to teach the model problem solving, and result of such training was measured by comparing the reference code with model generated code against series of Python coding tasks.

## **Potential Applications**

This project has various practical benefits:

- IDE Integration: Model could be locally integrated with IDE to provide offline coding assistance to users.
- Data Privacy: Sensitive code and data are processed completely on-site which preserves the users data privacy.
- Debugging Assistant: Assist to find the errors in the existing code and help to correct those errors to improve code.
- Code optimization: Removal of redundancy and suggests efficient algorithms to improve existing code.
- Educational assistant: Student can generate code for various standard algorithmic problems to understand and analyze the solution.



## **2 LITERATURE REVIEW**

### **2.1 Transformer Architecture and Attention Mechanism**

Transformer architecture is the foundation of all modern LLMs, which was introduced by Vaswani et.al [7]. Encoder decoder architecture with self-attention mechanism in the paper "Attention is all you need" is the basis of all modern LLMs, this revolutionary approach has become the standard natural language processing and remains the core architecture for model discussed in review.

### **2.2 Scaling Laws and Transfer Learning**

Neural network scaling law is important to develop efficient model, it is fundamental law explained by Kaplan et al. [1] for natural language model. Scaling laws provide insights on how model improves its performance with increased datasets, parameters and compute resources. In addition to that, Howard and Ruder [3] introduced Universal Language Model Fine-tuning (ULMFiT) which shows that transfer learning from pre-trained model can achieve result comparable to state-of-the-art model on text classification with small labeled dataset.

### **2.3 Few shot learning and in context learning**

It was demonstrated by Brown et al. [8] that large language models exhibit outstanding few shot learning capabilities, with GPT-3 and showing that model with enough scale can perform new tasks just from few examples without parameter updates. LLM can be adapted for various downstream tasks with the help of this fundamental capability.

### **2.4 LoRA**

LoRA was introduced by Hu et al. [5], which freezes weights of pre-trained model and injects trainable rank decomposition matrices into each layer of transformer. LoRA decreases the size of trainable parameters by 10000 times and GPU requirement by 3 times compared to full fine-tuning of GPT-3, which enables efficient adaptation on modest hardware.

### **2.5 QLoRA**

Quantization was combined with Low Rank Adaptation extended LoRA as per Dettmers et al [4], it introduced 4-bit Normalized Float (NF4), Double Quantization, and Paged Optimizers for memory reduction. This approach was able to achieve 99.3% of ChatGPT's performance with 24 hours of single-GPU training which demonstrated that high-quality small dataset is enough to achieve state-of-the-art results.

## 2.6 PEFT

Parameter-Efficient Fine-tuning for NLP through adapter modules was introduced by Houlsby et al. [9], showing that significant performance can be achieved by training only a fraction of original parameters of the model. Also, DoRA (Weight-Decomposed Low-Rank Adaptation), has been proposed by Liu et al. [10] recently which works on the basis of weight decomposition.

## 2.7 Open LLMs

Zhang et al. [11] released Open Pre-trained Transformers (OPT), which is one of the open-source LLMs released recently for public use and research, it is suite of decoder-only pre-trained models ranging from 125M to 175B parameters. Examples of such open-source models are, Touvron et al. [2] which introduced Llama-2, Roziere et al. [12] presented Code Llama which is a code-specialized family derived from Llama 2 with variants for general code completion. Comprehensive benchmarking of Llama-2 7B could be found on Diehl et al. [6], the conclusion derived from benchmark was that model performed well on simpler task but struggled with complex, parallelized computations.

## 2.8 Domain Specific Application

Fine-tuning Code Llama-7B for Fortran code generation [13] practically validated the PEFT techniques where LoRA was used to adapt attention weights freezing MLP modules, researchers gained significant improvements in compilation and execution of LeetCode problems, performing better than vanilla CodeLlama-7B-Instruct.

## 2.9 Evaluation metrics for code generation

Papineni et al. [14] introduced BLEU (Bilingual Evaluation Understudy), a popular automatic evaluation metric. Although, BLEU provides standardized evaluation for Natural Language Processing, other metrics such as CodeBLEU and human-eval are mostly used for comprehensive code related evaluation.

## 2.10 Multilingual code generations benchmarks

Benchmarking multilingual code generation is still a huge challenge as most of the evaluation methods primarily focus on English language. Peng et al. [15] introduced HumanEval-XL, a human-eval extension with multilingual capabilities. This benchmark evaluates whether the model is able to generate code coherently or not when it is given the instructions in multiple language.

### **2.11 Training with noisy labels**

Generalized Cross Entropy Loss for training deep neural networks with noisy labels was introduced by Zhang and Sabuncu [16], this technique is highly relevant when training a model on synthetic or raw scrape datasets, where label noise is high and robustness needs to be maintained for reliability of the model.

### 3 REQUIREMENTS ANALYSIS

#### 3.1 Hardware requirements

To match the demand of Llama-2-7B architecture, cloud services like google colab and Kaggle was utilized.

Table 3-1 Infrastructure Allocation and Hardware Resources

| Task              | Platform       | Hardware Resources             |
|-------------------|----------------|--------------------------------|
| Data Preparation  | Kaggle / Colab | CPU / T4 GPU (16GB VRAM)       |
| HPO (Optuna)      | Kaggle         | Dual T4 GPUs / P100            |
| Final Fine-tuning | Kaggle         | Dual T4 GPUs                   |
| Evaluation        | Kaggle         | Dual T4 GPUs / P100            |
| Inference         | Local (Ollama) | Consumer GPU (RTX 3060 / 4060) |

#### 3.2 Software requirements

For memory-efficient training and benchmarking purposes ecosystem of various python libraries were used.

##### 3.2.1 Programming language

Python 3.8: It is the primary language used because of its extensive support for machine learning and data science.

##### 3.2.2 Frameworks and libraries

- Numpy and Pandas: This library helps in preprocessing and efficient data manipulation for handling prompts, predictions, and evaluation metrics. See: <https://numpy.org/>, <https://pandas.pydata.org/>.
- BitsAndBytes : This library provides 4-bit Quantization (NF4) and Double Quantization which enables the model to fit in limited GPU memory. See:<https://github.com/bitsandbytes-foundation/bitsandbytes>.
- Transformers and TRL: These are Hugging Face libraries used to load, tokenize, and generate code using already trained language models. The fine-tuning loops are managed using Trainer API. See: <https://huggingface.co/docs/transformers/index>, <https://huggingface.co/docs/trl/index>.
- Optuna : TPE (Tree-structured Parzen Estimator) can be used for Hyperparameter optimization framework for automatical search of optimal learning rate, LoRA alpha, dropout, and other tuning parameters. See: <https://optuna.org/>

- PEFT: It helps to implement LoRA adapters, reducing trainable parameters meanwhile effective adaptation of large language model is allowed. See: <https://huggingface.co/docs/peft/index>.
- Datasets: It allows easy access to various datasets such as human-eval, enabling straightforward integration with various libraries such as PyTorch. See: <https://huggingface.co/docs/datasets/index>, [https://huggingface.co/datasets/openai/openai\\_humaneval](https://huggingface.co/datasets/openai/openai_humaneval)
- PyTorch: It is deep learning framework which is used for training of model, GPU acceleration, automatic differentiation, and management of tensor operations during various process like fine-tuning and evaluation. See: <https://pytorch.org/>.
- Human-eval: It is a standard datasets used during evaluation process of large language models. In this project it is used to compute pass@k metrics. See: [https://huggingface.co/datasets/openai/openai\\_humaneval](https://huggingface.co/datasets/openai/openai_humaneval).
- TQDM: It is used to provide progress bar for loops to visually track iterations. See: <https://tqdm.github.io/>.
- Matplotlib: It is used to visualize training metrics, validation loss, and evaluation results for various large language models. See: <https://matplotlib.org/>.
- CodeBLEU: It is metric to check code generation quality considering syntax, data-flow, and n-gram similarity. See: <https://github.com/k4black/codebleu>.
- Sacrebleu: It is standardized BLEU score computation for evaluation of code similarity and quality of translation for generated solutions. See: <https://github.com/mjpost/sacrebleu>.

### 3.2.3 Development Environment

- Hugging Face Hub: It is platform for sharing and accessing pre-trained models and datasets in one place. See: <https://huggingface.co/>.
- Google Colab: It is a cloud platform which provides CPU and GPU to users for training and finetuning language model. See: <https://colab.research.google.com/>.
- Kaggle: It provides powerful CPU and GPU such as double T4 for training and hyperparameter tuning. See: <https://www.kaggle.com/>.
- Weights and Biases (W&B): It is used for monitoring validation loss curves, and logging hardware consumption across various trials of training and hyperparameter tuning. See: <https://wandb.ai/site>.
- Git & GitHub: It is version control and collaboration platform for developers to do code management and documentation. See: <https://git-scm.com/>, <https://github.com/>.
- LaTeX: It is used for writing the reports with professional formatting and presentation of results. See: <https://www.latex-project.org/>.

### **3.3 Feasibility Analysis**

This project is primarily targeted at fine-tuning large language models for Python code generation using Parameter Efficient Fine-Tuning (PEFT) with QLoRA. The feasibility of this project is analyzed from the following perspectives

#### **3.3.1 Schedule Feasibility**

The project is scheduled into three phases including data collection, model-fine tuning, and evaluation. With the completion of evaluation before the final stage the project is well-aligned with the academic calendar.

#### **3.3.2 Operational Feasibility**

The project is designed to be applicable on relatively low-end devices. The hardware requirement for Llama-2 7B model is approximately 6 to 8 GB of VRAM, which is widely available in consumer-grade GPUs such as the NVIDIA RTX 3060 or 4060 and the Google Colab free tier (T4 GPU). The use of QLoRA significantly reduces the memory footprint, making it feasible to perform fine-tuning on a single consumer-grade GPU rather than requiring an enterprise-level server cluster.

#### **3.3.3 Economic Feasibility**

The project utilizes freely available open-source models (Llama-2) and public datasets (FlyTec, Alpaca). Since the hardware constraints are low enough to utilize existing consumer-grade GPUs or free cloud-based services like Google Colab, the primary development cost is minimal.

#### **3.3.4 Technical Feasibility**

Implementation of this project is dependent on industry-standard libraries such as Hugging Face transformers, peft, and bitsandbytes, and these library provides integration of 4-bit quantization and LoRA adapter. Our team has the required knowledge for managing the model loading, data preprocessing, and evaluation pipelines effectively in Python programming and machine learning frameworks.

#### **3.3.5 Legal Feasibility**

The project follows all the rules and does what is right. It uses open-source models and datasets so it does not break any rules about who can use them. Also, the project does not use any information that can be used to identify a person or any sensitive information so peoples privacy is safe. All the code and documentation for the project will be shared with everyone under an open-source license so that people can see what is going on and

work together on the project as the project is about being open and working together with the research community, on the project.

## 4 SYSTEM ARCHITECTURE AND METHODOLOGY

### 4.1 System Architecture of LLaMA-2

LLaMA-2, which is a part of Large Language Model Meta AI is a group of language models made by Meta and they are built to generate human-like text. This is done by studying large amount of text to find patterns. This model comes in sizes with range between 7 billion and 70 billion parameters. These models are made to work for both studying and real-life uses and are used in a lot for things like creating text answering questions, summarizing pieces of writing and generating code because they are easy to get and work well.

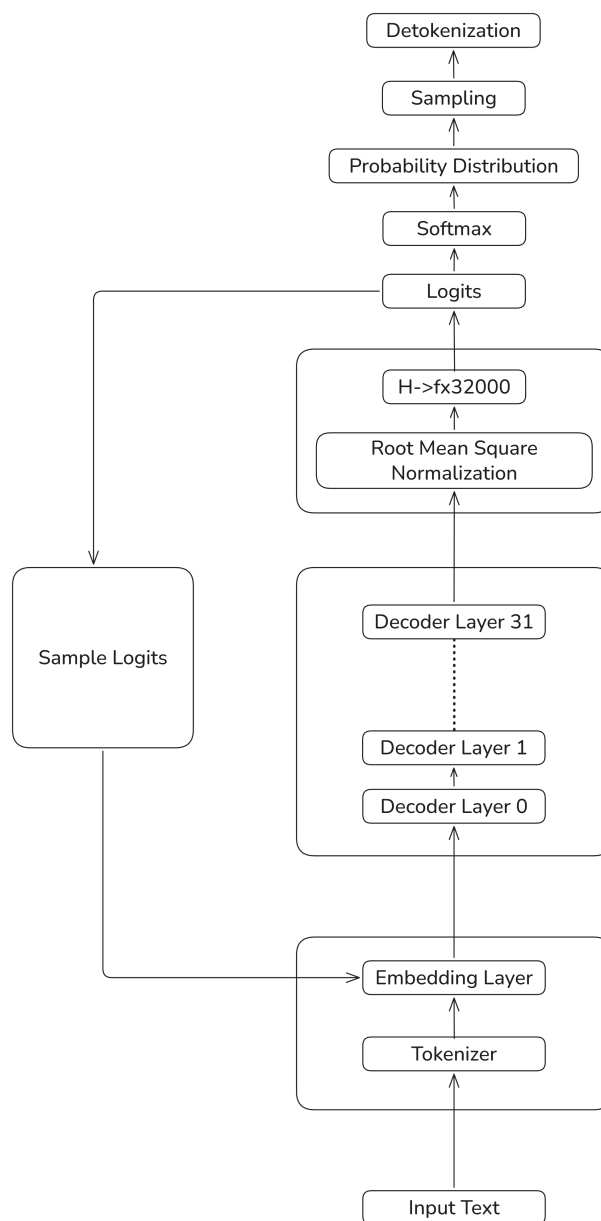


Figure 4-1 LLaMA-2 Architecture



#### 4.1.1 LLaMA Architecture Variation

In this model, there are various decoder layers and each layers contains attention mechanism which is followed by position wise feed-forward neural network, and residual connections and layer normalization are present all over neural network to improve training stability and efficient gradient flow.

Table 4-1 Architectural Variants of LLaMA-2 Models

| Model Size  | Parameter Size | Training Tokens | Attention Heads | Layers |
|-------------|----------------|-----------------|-----------------|--------|
| LLaMA-2 7B  | 7 Billion      | ~2 Trillion     | 32              | 32     |
| LLaMA-2 13B | 13 Billion     | ~2 Trillion     | 40              | 40     |
| LLaMA-2 70B | 70 Billion     | ~2 Trillion     | 64              | 80     |

Table 4-2 Context Length of LLaMA-2 Models

| Model Size  | Context Length (Tokens) |
|-------------|-------------------------|
| LLaMA-2 7B  | 4096                    |
| LLaMA-2 13B | 4096                    |
| LLaMA-2 70B | 4096                    |

#### 4.1.2 LLaMA-2 Architecture Analysis

The LLaMA-2 model uses a kind of design called a decoder-only transformer which is made for language modeling and this model tries to guess the next word. The decoder-only design processes input words one by one, it looks at all the words through a special mechanism called self-attention and this mechanism is used to make sure that it only looks at the words that come before it which helps the model to generate text and code. Each part of the decoder has two components, one is a -head self-attention module and the other is a feed-forward network that works on each position. Also, the multi-head attention helps the model to understand relationships between words, it does this by looking at information from angles. This model also has connections and layer normalization, these help to make the training process stable and improve performance.

In short LLaMA-2 uses a decoder- transformer design which has masked -head self-attention and other features and these features make LLaMA-2 good for large-scale language modeling and generation tasks. The LLaMA-2 model is suitable, for text and code generation tasks.

#### 4.1.3 Pre-training of LLaMA-2

The training of LLaMA-2 was done using a collection of data from public sources which was made up of a lot of things. This training collection had two trillion pieces of information. The people who made LLaMA-2 picked this collection carefully wanted to

make sure it was good and did not cost much. Also, to make LLaMA-2 better at giving information and to stop it from making things up the people who made it used more reliable sources when they were training it.

The way LLaMA-2 was trained is by looking at a sequence of information and trying to guess what comes next and this was mainly how LLaMA-2 learnt and it is good way of training for model like LLaMA-2 as it helps LLaMA-2 to be very good at generation of next token. Meta trained LLaMA-2 on variety of datasets to make it more general model.

#### **4.1.4 Chat Model of LLaMA 2**

The specialized version of base LLaMA 2 large language model developed by Meta AI for conversational tasks is LLaMA-2-7B-chat. This chat model is fine-tuned primarily on dialogue interactions dataset . So, it is basically a base mode fine-tuned on chat format to make it easier for normal users to interact.

This chat variant was trained using Supervised Fine-Tuning (SFT) followed by Reinforcement Learning with Human Feedback (RLHF) and in supervised fine-tuning, the model learns from good quality example conversations written by human annotators. After that, reinforcement learning is applied where human evaluators rank various responses and giving higher reward to high ranked responses promoting model generate more such responses.

The format of the conversation is:

```
<s>[INST] Explain recursion in simple terms. [/INST]
Recursion is a programming technique where a function calls
↳ itself to solve smaller parts of the same problem.
</s>
```

This specific format helps model to distinguish between the instruction and responses during training and inference.

#### **4.1.5 Laws of scaling [1]**

Law of scaling describes how the performance of model changes when three primary aspects are scaled:

1. Model size (parameters)
2. Dataset size (training tokens/examples)
3. Compute (FLOPs / training steps)

The core idea, originally discovered in neural language models, was "When you increase model size, data size, and compute in suitable proportions, error decreases according to a power-law relationship [1]".

### Mathematical Formulation of Scaling Laws

Power law relationship is used to predict the performance of language models which involves model size, data size and compute. It can be expressed as:

$$\mathcal{L}(N) = \mathcal{L}_\infty + \frac{A}{N^\alpha} \quad (4-1)$$

where:

- $\mathcal{L}(N)$  is the training or validation loss as a function of model size,
- $N$  represents the number of model parameters,
- $\alpha$  is the scaling exponent,
- $A$  is a constant,
- $\mathcal{L}_\infty$  denotes the irreducible loss.

As per the given equation of a power-law decay, as the number of parameters increases, the loss decreases.

Also scaling with respect to total training compute (measured in FLOPs) follows:

$$\mathcal{L}(C) = \mathcal{L}_\infty + \frac{B}{C^\beta} \quad (4-2)$$

where  $\beta$  is the compute scaling exponent and  $B$  is a constant.

### Log-Log Linear Relationship

When plotted in log-log space, scaling laws exhibit an approximately linear relationship:

$$\log \mathcal{L} = -\alpha \log N + \text{constant} \quad (4-3)$$

This linear behavior in logarithmic space confirms the power-law nature of neural scaling.

### Data-Model Scaling Trade-off

Dataset size  $D$  also has impact on performance improvements, so, if model size is increased without the increase in dataset size then it may lead to serious problems like

over-fitting. Empirical evidences suggests that model parameters and training tokens should be scaled together to have the optimal performance of the model as:

$$N \propto D \quad (4-4)$$

This balanced scaling ensures efficient utilization of compute resources and prevents under-training of large models.

### **Implications for Fine-Tuning**

Similar principles of scaling law, described for the full pre-training, could be used for fine-tuning. For parameter-efficient methods such as QLoRA, we use low-rank updates approximate task-specific adaptations in a reduced subspace not scaling all mode parameters which shows that model performance could be improved provided that there is sufficient high-quality task-specific datasets.

### **Practical Implications for This Study**

Due to computational constraints the base mode is not scaled in parameter size in this project. So, performance was improved through:

- Increasing dataset diversity and quality,
- Efficient compute allocation during fine-tuning,
- Low-rank parameter adaptation via QLoRA.

According to scaling law principles, improving data quality and maintaining balanced compute usage is expected to yield more stable performance gains than increasing model parameters under limited hardware resources.

#### **4.1.6 Training Configuration**

The pre-training setup and architectural design of LLaMA-2 is similar to that of LLaMA-1 with several key improvements, This model uses a standard transformer architecture where pre-normalization is done via RMSNorm, Feed-Forward layers uses the SwiGLU activation function, and rotary positional embeddings (RoPE) are used to encode positional information. The increased context length to 4096 tokens from LLaMA-1 is one of the fundamental architectural improvement.

#### **4.1.7 AdamW Optimizer**

New AdamW optimizer is a variation of original Adam optimization algorithm which incorporates decoupled weight decay regularization in Adam optimization. And

this AdamW optimizer is designed to improve generalization performance with the application of weight decay directly into the parameters rather than the gradients, which leads to better regularization and better convergence properties. The rule for AdamW update is expressed in Equation (4-5):

$$\theta_{t+1} = \theta_t - \eta \left( \frac{m_t}{\sqrt{v_t} + \epsilon} + \lambda \theta_t \right) \quad (4-5)$$

where  $\theta_t$  represents the model parameters at time  $t$ ,  $\eta$  is the learning rate,  $m_t$  and  $v_t$  are the first and second moments of the gradients,  $\epsilon$  is a small constant for numerical stability, and  $\lambda$  is the weight decay coefficient.

AdamW is effective in training large language model which provides better convergence and generalization as compared to original Adam, mainly during training of large batch sizes or large datasets.

#### 4.1.8 Cosine Learning Rate Scheduler

The cosine learning rate scheduler is a way to change the learning rate during the model training, and it does this by following a pattern that looks like a cosine curve which helps to make the training process more stable. The cosine learning rate scheduler updates the learning rate based on equation (4-6):

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left( 1 + \cos \left( \frac{\pi t}{T} \right) \right) \quad (4-6)$$

where  $\eta_{\max}$  and  $\eta_{\min}$  are the maximum and minimum learning rates,  $t$  is the current iteration, and  $T$  is the total number of iterations.

This method decreases the learning rate gradually which helps the model to converge more smoothly and avoid getting stuck in local minima. This method is more effective when it is combined with warm-up strategies.

#### 4.1.9 Input Sequence Representation

The conversational context and safety constraints are maintained during inference, a structured format is followed in LLaMA-2-7B-chat which segregates system instructions, user inputs, and output of the model.

The general input sequence  $\mathbf{X}$  is represented in (4-7):

$$\mathbf{X} = \langle s \rangle [\text{INST}] \ll \text{SYS} \gg S \ll / \text{SYS} \gg I [/ \text{INST}] R \langle /s \rangle \quad (4-7)$$

Where the components are defined as follows:

- $\langle s \rangle$ : The Start-of-Sequence (SOS) token.
- [INST]: Special tag indicating the beginning of a user instruction.
- $\ll \text{SYS} \gg S \ll / \text{SYS} \gg$ : The system prompt  $S$ , which defines the model's persona or operational constraints.
- $I$ : The specific instruction or query provided by the user.
- [/INST]: Special tag indicating the end of the instruction block.
- $R$ : The response or target output generated by the model.
- $\langle /s \rangle$ : The End-of-Sequence (EOS) token.

### Practical Example of Sequence Construction

To demonstrate the application of the LLaMA-2 chat template, consider a scenario involving a system prompt  $S$ , a user instruction  $I$ , and the resulting model response  $R$ .

Let:

- $S = \text{"You are a Python code assistant."}$
- $I = \text{"Write a function to compute factorial."}$
- $R = \text{"def factorial(n): ... "}$

The complete input sequence  $\mathbf{X}$  is constructed by concatenating these components within the specialized control tokens as shown in 4-8.

$$\begin{aligned} \mathbf{X} = & \langle s \rangle [\text{INST}] \ll \text{SYS} \gg \text{You are a Python code assistant.} \ll / \text{SYS} \gg \\ & \text{Write a function to compute factorial.} [/ \text{INST}] \\ & \text{"def factorial(n): ..."} \langle /s \rangle \end{aligned} \quad (4-8)$$

#### 4.1.10 Tokenizer and Vocabulary

Byte Pair Encoding (BPE) is used in LLaMA-2 as tokenizer which is implemented via SentencePiece library, and it is the same tokenizer used in LLaMA-1. This tokenizer handles the numerical values by splitting them into individual digits and decomposing unknown UTF-8 characters into byte-level representations ensuring full coverage and the resulting vocabulary consists of 32,000 unique tokens..

#### Byte Pair Encoding (BPE) Mechanism

Let us consider a simple corpus containing the words: low, lower, newest, and widest. This BPE algorithm iteratively merges the most frequent adjacent pairs of characters

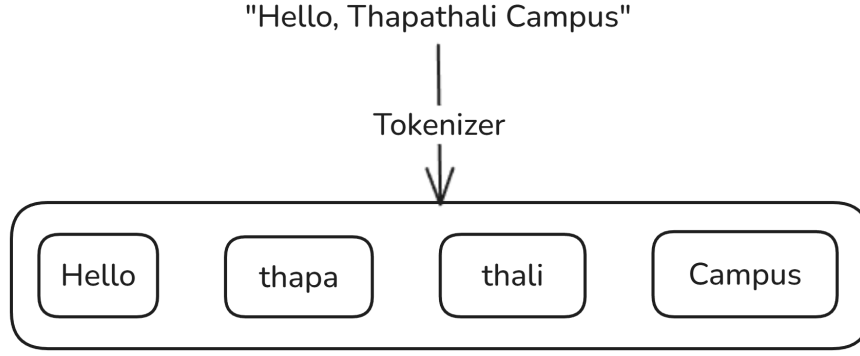


Figure 4-2 Tokenization Process

or subwords to form new tokens from them and this step-by-step merging process for corpus is detailed in 4-3.

Table 4-3 Step-by-Step BPE Token Merges

| Step | Symbol Pairs Merged | Resulting Token Sequence                                 |
|------|---------------------|----------------------------------------------------------|
| 0    | -                   | l o w, l o w e r, n e w e s t, w i d e s t               |
| 1    | e s                 | l o w, l o w e r, n e e s t, w i d e e s t               |
| 2    | e s t               | l o w, l o w e r, n e e s t, w i d e e s t               |
| 3    | l o                 | <b>l o</b> w, <b>l o</b> w e r, n e e s t, w i d e e s t |
| 4    | w e                 | l o <b>w e</b> r, n e e s t, w i d e e s t               |

When several iteration are done, frequently occurring subword units such as *lo*, *we*, and *est* are merged which results in a subword vocabulary efficiently represent all words in the corpus, including rare or unseen words, by breaking them down into known sub-components. “Out-of-Vocabulary” (OOV) problem which is common in Natural Language Processing is reduced by the use of this approach..

#### 4.1.11 Embedding Layer

Relationship between tokens based on their similarity are captured by the embedding layer which servers as the foundation for future processing in transformer. To capture semantic and syntactic information, each token is mapped to an embedding vector of dimension  $d$ . The embedding matrix  $\mathbf{E} \in \mathbb{R}^{|V| \times d}$  is learned during training, where  $|V|$  is the vocabulary size. Given an input token sequence  $\mathbf{X} = [x_1, x_2, \dots, x_n]$ , it is converted into an embedding matrix  $\mathbf{X}_e \in \mathbb{R}^{n \times d}$  by indexing into the embedding matrix:

$$\mathbf{X}_e = \mathbf{E}[x_1, x_2, \dots, x_n] \quad (4-9)$$

The embedding layer captures relationships between tokens based on similarity and serves as the foundation for further processing in the transformer.

#### 4.1.12 RMSNorm

RMSNorm (Root Mean Square Layer Normalization) stabilizes the training process and accelerate convergence [17]. And it is unlike standard Layer Normalization, which uses mean subtraction. This method achieves invariant re-scaling by doing the root mean square of activations which allows similar performance with lower computational overhead.

For an input vector  $\mathbf{x} \in \mathbb{R}^d$ , RMSNorm first computes the root mean square:

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} \quad (4-10)$$

The normalized output is then given by:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \cdot g \quad (4-11)$$

where  $g \in \mathbb{R}^d$  is a learned gain parameter. The robust gradient flow and stable training in deep neural network is maintained in LLaMA-2 with the help of pre-normalization through RMSNorm, while doing this it remains more efficient than older LayerNorm.

#### 4.1.13 Multi-Head Attention

The fundamental attention mechanism used to capture complex relationships between tokens is Multi-Head Attention (MHA) which allows the mode to jointly attend information from various subspaces of representation at various positions.

Given input embeddings  $\mathbf{X} \in \mathbb{R}^{n \times d}$ , where  $n$  is sequence length and  $d$  is hidden dimension, the input is projected into query, key, and value matrices:

$$\mathbf{Q} = \mathbf{X}W^Q, \quad \mathbf{K} = \mathbf{X}W^K, \quad \mathbf{V} = \mathbf{X}W^V \quad (4-12)$$

where  $W^Q, W^K, W^V \in \mathbb{R}^{d \times d_k}$  are learned weight matrices and  $d_k = d/h$  is the dimension of each of the  $h$  attention heads.

The outputs from all heads are concatenated and linearly transformed using an output projection matrix  $W^O \in \mathbb{R}^{d \times d}$ :

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (4-13)$$



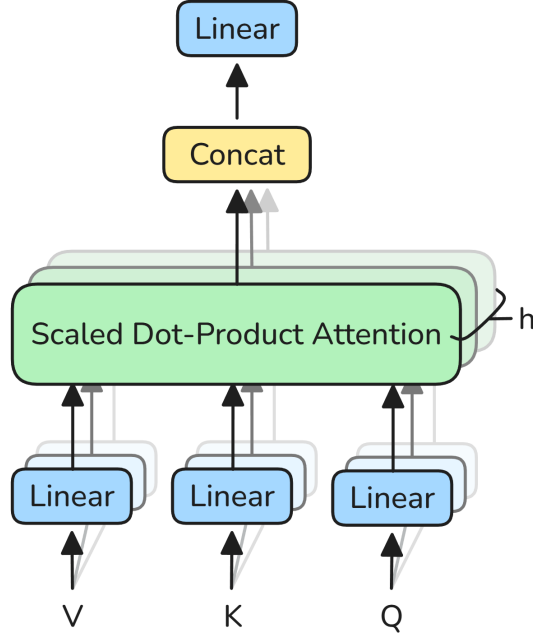


Figure 4-3 Multi-Head Attention Mechanism

This parallel processing captures syntactic dependencies and contextual nuances more effectively than single-head attention.

#### 4.1.14 Rotary Positional Embeddings (RoPE)

Equation (4-4) of rotation is used to encode relative positions between tokens, and improves the ability of model to generalize for the sequence which are longer than those which are seen during the training process.

The rotation is represented through sine and cosine functions, with a complex-number interpretation. RoPE is effective because it helps the model encode relative positional relationships implicitly and can generalize beyond the context window used during training.

#### Mathematical Formulation

Let  $\mathbf{q}, \mathbf{k} \in \mathbb{R}^d$  denote the query and key vectors of a transformer attention layer, and let  $p$  be the token position. RoPE partitions the hidden dimensions into pairs and applies a rotation to each pair  $(q_{2i-1}, q_{2i})$  and  $(k_{2i-1}, k_{2i})$  using the following transformation:

$$\begin{bmatrix} q'_{2i-1} \\ q'_{2i} \end{bmatrix} = \begin{bmatrix} \cos(p\theta_i) & -\sin(p\theta_i) \\ \sin(p\theta_i) & \cos(p\theta_i) \end{bmatrix} \begin{bmatrix} q_{2i-1} \\ q_{2i} \end{bmatrix} \quad (4-14)$$

Similarly, the keys are rotated as:

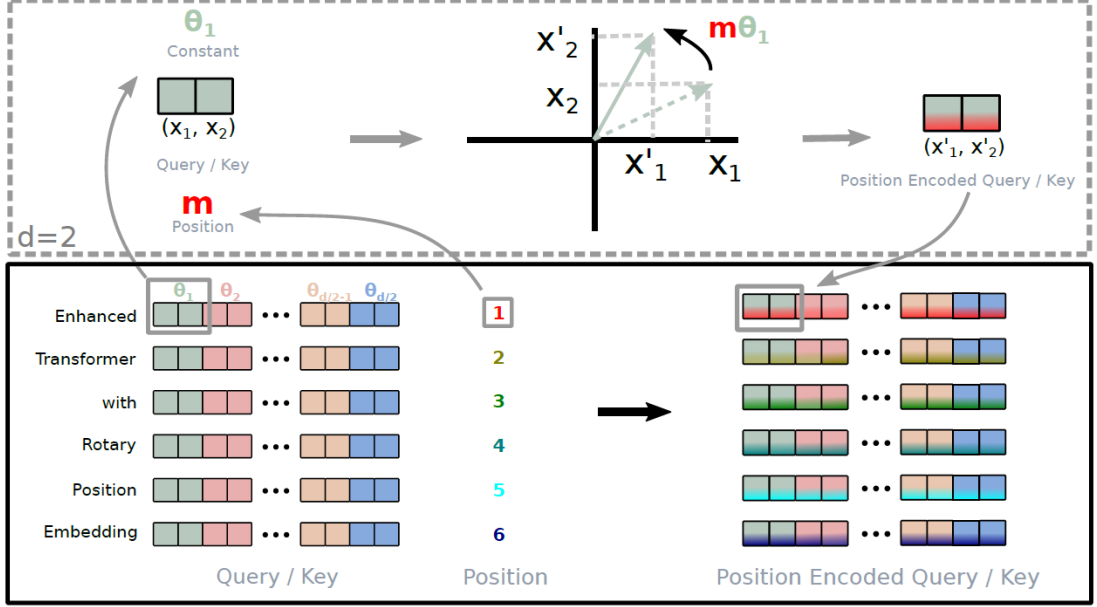


Figure 4-4 Rotary Positional Embeddings

$$\begin{bmatrix} k'_{2i-1} \\ k'_{2i} \end{bmatrix} = \begin{bmatrix} \cos(p\theta_i) & -\sin(p\theta_i) \\ \sin(p\theta_i) & \cos(p\theta_i) \end{bmatrix} \begin{bmatrix} k_{2i-1} \\ k_{2i} \end{bmatrix} \quad (4-15)$$

In these equations,  $\theta_i$  represents the precomputed frequency for the  $i$ -th dimension pair, defined as:

$$\theta_i = 10000^{-2(i-1)/d} \quad (4-16)$$

Following the rotation, the transformed query matrix  $\mathbf{Q}'$  and key matrix  $\mathbf{K}'$  are used to compute the scaled dot-product attention:

$$\text{Attention}(\mathbf{Q}', \mathbf{K}', \mathbf{V}) = \text{softmax} \left( \frac{\mathbf{Q}'(\mathbf{K}')^\top}{\sqrt{d_k}} \right) \mathbf{V} \quad (4-17)$$

This rotational formulation helps the attention mechanism encode relative positions between tokens, improving the model's ability to generalize to sequences longer than those seen during training.

#### 4.1.15 Feed-Forward Network

In each decoder layer of LLaMA-2 feed-forward network are present which improves the model's ability to capture non-linearity, this feed-forward network contains two linear transformations with activation function in between:

$$\text{FFN}(\mathbf{x}) = \text{SwiGLU}(\mathbf{x}W_1)W_2 \quad (4-18)$$

where  $W_1, W_2 \in \mathbb{R}^{d \times d_{ff}}$  are learned weight matrices, and  $d_{ff}$  is the intermediate hidden dimension, often set larger than  $d$  (e.g.,  $d_{ff} = 4d$ ).

The improvement of model expressivity and training stability is due to the use of SwiGLU activation function, and feed-forward network enables parallel computation while strengthening overall representational power.

#### 4.1.16 Activation Function (SwiGLU)

SwiGLU activation function is a variant of the Gated Linear Unit which improves model expressivity and training stability [18] as compare to older ReLU activations, and this activation function is integrated in feed-forward network of each decoder layer.

For an input vector  $\mathbf{x} \in \mathbb{R}^d$ , the SwiGLU mechanism first computes two distinct linear projections:

$$\mathbf{a} = \mathbf{x}W_1, \quad \mathbf{b} = \mathbf{x}W_2 \quad (4-19)$$

where  $W_1, W_2 \in \mathbb{R}^{d \times d_{ff}}$  are learned weight matrices, and  $d_{ff}$  denotes the intermediate hidden dimension of the feed-forward layer.

The output of the SwiGLU layer is then determined by the element-wise multiplication of the first projection with the gated second projection:

$$\text{SwiGLU}(\mathbf{x}) = \mathbf{a} \odot \text{SiLU}(\mathbf{b}) \quad (4-20)$$

where  $\odot$  denotes the Hadamard (element-wise) product. The SiLU (Sigmoid Linear Unit) function, also known as the swish activation, is defined as:

$$\text{SiLU}(z) = z \cdot \sigma(z) = \frac{z}{1 + e^{-z}} \quad (4-21)$$

By gating the linear transformation  $\mathbf{a}$  with the non-linear SiLU output of  $\mathbf{b}$ , SwiGLU helps the network model complex patterns effectively through controlled information flow.

#### 4.1.17 Softmax Output Layer

The final output layer of the LLaMA-2 model is a linear transformation followed by a softmax function, which converts the model's hidden state into a probability distribution over the vocabulary for token prediction. Here the output logits are computed from output of the last transformer decoder layer  $h \in \mathbb{R}^{n \times d}$ , where  $n$  is the sequence length and  $d$  is the hidden dimension as shown in equation (4-22):

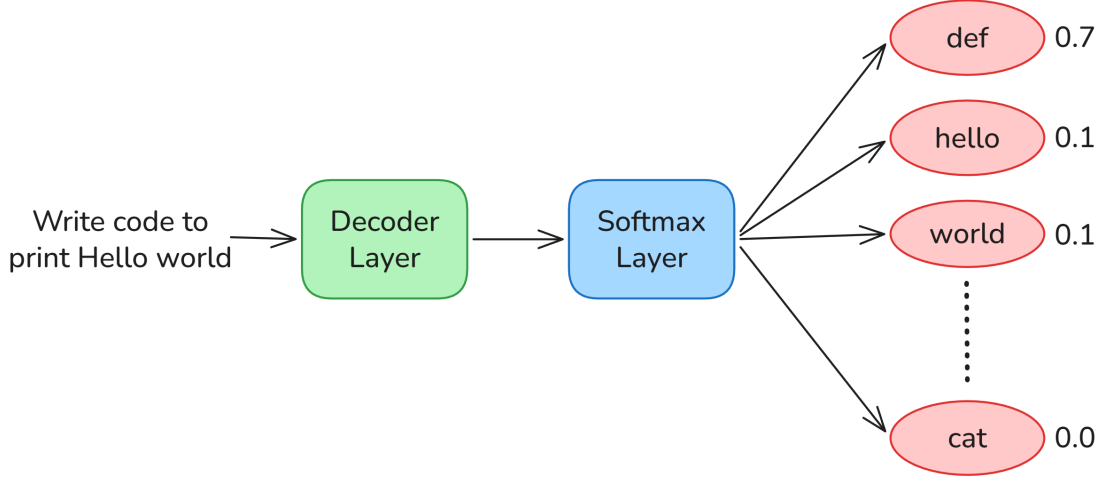


Figure 4-5 Softmax Output Layer

$$\mathbf{z} = \mathbf{h}W^{\text{out}} + b^{\text{out}} \quad (4-22)$$

where  $W^{\text{out}} \in \mathbb{R}^{d \times V}$  is the output weight matrix and  $b^{\text{out}} \in \mathbb{R}^V$  is the output bias vector, and  $V$  is the size of the vocabulary.

We compute the final output probabilities using equation (4-23):

$$P(x_t = v) = \frac{\exp(z_v)}{\sum_{v' \in V} \exp(z_{v'})} \quad (4-23)$$

where  $z_v$  is the logit corresponding to token  $v$  in the vocabulary. This softmax layer allows the model to generate a probability distribution over all possible next tokens, enabling it to perform autoregressive generation by sampling or selecting the token with the highest probability at each step.

#### 4.1.18 Sampling Strategies

During inference, LLaMA-2 generates text by sampling from the probability distribution produced by the softmax output layer. We can use following strategies to control the diversity of the generated text:

## **Greedy Sampling**

It is the simplest strategy where the model selects the token with highest probability from generated probability distribution. Using this approach causes deterministic and repetitive behavior, as it doesn't include alternative tokens which may have slightly lower probabilities.

## **Top-k Sampling**

In top-k sampling, the sampling is restricted to top k most probable tokens. This model improves the randomness in text generation as model can output less probable tokens also.

## **Nucleus Sampling (Top-p Sampling)**

Nucleus sampling limits the sampling to top tokens whose probability sums up to a threshold  $p$ . This method provides a dynamic control over the range of tokens instead of fixed number of tokens. we can make choice between generating more logical text and more creative text can be obtained by setting lower  $p$  and higher  $p$  respectively.

## **Temperature Scaling**

Temperature is used to adjust the actual probability distribution before sampling occurs. It controls the randomness of output by first dividing the logits by temperature parameter  $T$ . If higher-probability tokens are to be favored a lower temperature is set which makes the distribution sharper. And if diversity in tokens are to be favored a higher temperature is set which makes the distribution flatter.

## **4.2 Code Generation with LLMs**

It has been observed that Large language Models (LLMs) such as LLaMA-2 have robust code generation capabilities. These models are trained using large code datasets from which they learn programming syntax, semantics and structural patterns. This enables them to generate source code from natural language instructions, perform code completion, and assist in automated debugging.

### **4.2.1 Mathematical Perspective of Code Generation**

From the prospective of computation, code generation is treated as a conditional sequence generation problem. The process is decomposed into tokenization and autoregressive inference.

## Tokenization

We first process the instruction in natural language using tokenizers, such as Byte-Pair Encoding (BPE), which maps the input string to a sequence of discrete tokens:

$$\mathbf{x} = [x_1, x_2, \dots, x_n], \quad x_i \in V \quad (4-24)$$

where  $V$  is the model's vocabulary. Let's take the instruction "Write a Python function" as an example and transform into numerical vector:

$$\text{"Write a Python function"} \longrightarrow [101, 523, 324, 76, 981] \quad (4-25)$$

## Autoregressive Generation

The model is designed to generate code tokens sequentially using an autoregressive approach. The probability of predicting the next token  $x_{t+1}$  is conditioned on all previously generated tokens:

$$P(x_{t+1} \mid x_1, x_2, \dots, x_t) = \text{softmax} \left( \frac{\mathbf{Q}_t \mathbf{K}_t^\top}{\sqrt{d_k}} \right) \mathbf{V}_t \quad (4-26)$$

In this formulation:

- $\mathbf{Q}_t, \mathbf{K}_t, \mathbf{V}_t$  are the query, key, and value matrices computed within the transformer's self-attention layers at time step  $t$ .
- $x_{t+1}$  is the token with the highest predicted probability (or sampled) based on the preceding context  $[x_1, \dots, x_t]$ .

## Sequence Formation

After the token  $x_{i+1}$  is predicted it is appended to the existing sequence and then the process is repeated again:

$$\mathbf{x} \leftarrow [x_1, x_2, \dots, x_t, x_{t+1}] \quad (4-27)$$

After the token  $x_{i+1}$  is predicted it is appended to the existing sequence and then the process is repeated again:.

## Practical Generation Example

To illustrate the autoregressive process, consider a specific user instruction  $I$  and the corresponding model output  $R$ .

**Input Instruction:** *"Write a Python function to compute factorial"*

**Tokenized Input:**

$$[x_1, x_2, x_3, x_4, x_5] = [101, 523, 324, 76, 981] \quad (4-28)$$

**Step-by-step Token Generation:** The iterative prediction of subsequent tokens is represented in the following sequence:

$$\begin{aligned} x_6 &\sim P(x_6 | x_1, \dots, x_5) \longrightarrow \text{def} \\ x_7 &\sim P(x_7 | x_1, \dots, x_6) \longrightarrow \text{factorial} \\ x_8 &\sim P(x_8 | x_1, \dots, x_7) \longrightarrow (n) : \\ x_9 &\sim P(x_9 | x_1, \dots, x_8) \longrightarrow \text{if} \\ &\vdots \\ x_{15} &\sim P(x_{15} | x_1, \dots, x_{14}) \longrightarrow \text{return } n * \text{factorial}(n-1) \end{aligned} \quad (4-29)$$

**Generated Code Output:**

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n-1)
```

### 4.3 Dataset Collection and Preparation

It has been observed that the efficacy of a fine-tuned Large Language Model (LLM) is heavily dependent on the quality and diversity of the training corpus. For this project, we developed a multi-stage data engineering pipeline which transformed raw open-source data into a high-fidelity instructional corpus specialized for Pythonic reasoning.

#### 4.3.1 Data Source Synthesis

A combined data set containing data from two high-quality sources was constructed which provided a balance between general instruction following and deep algorithmic logic:

- **FlyTech Python Dataset (approx. 42k rows):** It has high quality dataset with good logical reasoning, mathematical problem solving, and functional Python snippets [19].

- **Alpaca-Style Code Instructions (approx. 18k rows):** It provides diverse instruction-following scenarios, ranging from simple automation scripting to complex software design patterns [20].

#### 4.3.2 Automated Classification and Quality Control

To ensure the model specialized in competitive programming rather than generic boilerplate code, we implemented an automated filtration layer. This prevented the model from learning suboptimal coding practices or irrelevant 'noise'.

1. **Syntactic Verification:** An initial script which utilizes Python's abstract Syntax Trees (AST) to parse every code was implemented. Here entries containing syntax errors or incomplete code blocks were discarded.
2. **Teacher-Model Classification:** We utilized the `Qwen2.5-Coder-1.5B` model as a zero-shot classifier. Labeling was based on the domain to which a particular problem belongs. We prioritized the "Algorithmic" subset to maximize the model's problem-solving density.
3. **Heuristic Pruning:** We removed entries with responses less than 7 words and repeat ratio greater than 0.4 to maintain a high information-to-token ratio.

#### 4.3.3 Justification for Model Selection: Qwen2.5-Coder-1.5B vs. Alternatives

The `Qwen2.5-Coder-1.5B` model is a specialized decoder-only language model from the Qwen-2.5 family. It was specially designed to provide high quality code. It's incorporation of state-of-the-art structural optimization allows it to rival larger general-purpose models in code classification and generation tasks despite of compact 1.54 billion parameter footprint. For the automated data cleaning and classification phase, we evaluated several candidate models. The selection of `Qwen2.5-Coder-1.5B` was based on a multi-dimensional trade-off between inference throughput, VRAM footprint, and code-specific reasoning accuracy.

Table 4-4 Comparison of Classification Engines for Data Preprocessing

| Model                     | Params      | VRAM (4-bit)   | Code Reasoning  | Tokens/Sec        |
|---------------------------|-------------|----------------|-----------------|-------------------|
| GPT-3.5-Turbo             | -           | Cloud-Based    | High            | Low (API Latency) |
| Llama-2-7B                | 7B          | ~5.2 GB        | Moderate        | ~45               |
| CodeLlama-7B              | 7B          | ~5.5 GB        | High            | ~42               |
| <b>Qwen2.5-Coder-1.5B</b> | <b>1.5B</b> | <b>~1.2 GB</b> | <b>Superior</b> | <b>~120+</b>      |

#### Technical Rationale

According to Table 4-4, despite having a higher number of parameters, bigger models like Llama-2-7B have a tendency towards generalization, which makes them less precise



in finding the special pattern of “Competitive Programming” compared to a particular “Coder” model.

1. **High-Throughput Batching:** The 1.5B model can perform at speeds of more than 120 tokens/second, allowing to complete the classification in significantly shorter time compared to the 7B model.
2. **Inductive Bias for Code:** Qwen2.5-Coder is pre-trained on a larger portion of source code as opposed to regular language compared to Llama-2, thus, has a stronger inductive bias towards finding programming logic as opposed to regular conversation.
3. **Local Privacy and Cost:** In order to avoid the need for using API calls (for example, OpenAI) for classification and ensure privacy, an instance of Ollama was used locally, eliminating token usage costs associated with classification of 60,000-row dataset.

This allowed to allocate more GPU power for the core fine-tuning process on the model.

#### 4.3.4 Data Formatting and Chat Templates

All samples were mapped in a structured format that takes advantage of the conversational capabilities of Llama-2. This helps the model understand when certain delimiters are used to transition from understanding the problem to generating solutions. The structure for Llama-2 looks like this:

```
<s>[INST]
{User_Instruction} [/INST]
{Python_Code_Solution} </s>
```

#### 4.3.5 Tokenization and Instruction Masking

Tokenization was performed using the Byte-Pair Encoding (BPE) tokenizer native to the Llama-2 model, and the maximum number of tokens per sequence was set to 4096. The key methodological point here is instruction masking. In particular, when calculating the loss during the SFT process, loss calculation was limited only to Python\_Code\_Solution tokens. For this purpose, 100 was used as an index of the labels for User\_Instruction tokens, which is the default ignore index in the PyTorch implementation of the CrossEntropyLoss function.

#### 4.3.6 Dataset Distribution

After shuffling and removing duplicate datapoints, the final dataset was divided into distinct subsets for training and evaluation:

- **Training Set (90%):** 12522 prompt-code pairs were used for training where 5% of the training data was reserved for validation.
- **Test Set (10%):** 1392 prompt-code pairs were reserved for post-training evaluation using the BLEU and CodeBLEU benchmark to measure the model’s generalization to unseen problems.

To maximize the available data for training while still retaining enough datapoints for unbiased evaluation a split of 90

In addition to this we used 164 unseen problems from Human-eval dataset for functional evaluation of the model’s code generation capabilities using pass@k metrics.

#### 4.4 Hyperparameter Tuning

Hyperparameters are critical part of a model training as hyperparameters control the stability and effectiveness of training. We can change values for learning rate, batch size, number of training epochs etc. to control how fast or slow a model learns with all the hyperparameters having their own pros and cons. The values for hyperparameters must be assigned before actual training begins instead being learned during training.

Formally, hyperparameter optimization can be expressed as:

$$\lambda^* = \arg \min_{\lambda \in \Lambda} \mathcal{L}_{val}(\lambda) \quad (4-30)$$

where  $\lambda$  represents the set of hyperparameters,  $\Lambda$  denotes the search space, and  $\mathcal{L}_{val}$  is the validation loss.

##### 4.4.1 Search Strategy

We employed the **Tree-structured Parzen Estimator (TPE)** for hyperparameter optimization. It is a Bayesian optimization method well-suited for mixed search spaces containing continuous, discrete and conditional parameters. Instead of modeling the objective directly as  $p(y | x)$  (where  $x$  is a hyperparameter configuration and  $y$  is validation loss), TPE models the inverse densities  $p(x | y)$  by splitting observed trials using a threshold  $y^*$ :

$$p(x | y) = \begin{cases} l(x), & y < y^* \\ g(x), & y \geq y^* \end{cases} \quad (4-31)$$

where  $l(x)$  represents configurations associated with better losses and  $g(x)$  represents the remaining configurations. New candidates are sampled from  $l(x)$  and ranked using the ratio  $\frac{l(x)}{g(x)}$ , which approximates maximizing expected improvement.

The TPE algorithm creates an appropriate balance between exploration and exploitation:

- **Exploration:** Ensures diversity by drawing samples from probabilistic density estimates over the whole search space.
- **Exploitation:** Focuses on areas with lower validation losses in the past by selecting higher values of  $\frac{l(x)}{g(x)}$ .

In recent studies, researchers have proven that TPE outperforms other search methods like grid search and random search in terms of faster convergence to optimal hyperparameters.

The following hyperparameters were optimized:

- **Learning Rate ( $\eta$ ):** Controls the size of the updates to the gradients, with significant effects on both the rate and stability of convergence.
- **Batch Size:** Governs the number of samples that will be used per optimization iteration and consequently affects the variance of the gradients and memory requirements.
- **LoRA Rank ( $r$ ):** Controls the dimensionality of the low-rank adaptation matrices, where higher ranks increase representation power at the cost of more parameters to optimize.
- **LoRA Dropout Rate:** Implements dropout regularization in the low-rank adaptation layers in order to address overfitting and improve generalization.
- **Learning Rate Scheduler Parameters:** Consists of hyperparameters like the number of warm-up steps, which control how the learning rate changes throughout training
- **Warm-up Ratio:** Determines the ratio of the total number of training steps allocated for gradually increasing the learning rate starting from a lower rate to a desired learning rate. This helps stabilize the training process at its initial stage.
- **LoRA Alpha ( $\alpha$ ):** The multiplier used on the low-rank matrices that controls the scale of parameter changes.

#### 4.4.2 Search Space Definition

The following describes how bounded spaces for searching are defined in order to encourage efficient searching:

- Learning rates are sampled logarithmically to encourage efficient exploration over a broad range of magnitudes.
- The batch size is selected from discrete feasible values based on GPU memory constraints.
- The LoRA rank is explored in a low-dimensional space in order to keep the number of parameters low..

- The dropout rate and warm-up ratio are sampled in continuous bounded ranges.

It should be mentioned that logarithmic sampling of the learning rate is especially important when using transformers for fine-tuning, as the optimal values typically differ by several orders of magnitude.

#### 4.4.3 Practical Considerations

Testing the hyperparameters on the whole data using a long training period would not be feasible since the training time required for large language models grows exponentially with respect to the size of the data and the model itself. Therefore, a subset of the training data was chosen, and the process of hyperparameter tuning followed. This results in significant savings in computational effort while maintaining an accurate estimate of the performance trends. The hyperparameter optimization methodology used in this study exhibited stability, better generalization capability, and efficient resource utilization.

### 4.5 Quantized Low-Rank Adaptation (QLoRA)

Parameter Efficient Fine-Tuning Methodology using Quantized Low Rank Adaptation (QLoRA) refers to a methodology for fine tuning a pre-trained model which employs the technique of combining low rank adaptation with a quantized and fixed set of model parameters. Through this methodology, one can drastically reduce the memory requirements of fine-tuning large models.

#### 4.5.1 Mathematical Formulation

The efficiency of QLoRA is derived from two primary mechanisms: weight quantization and the addition of trainable low-rank adapters.

##### Weight Quantization

Let  $\mathbf{W} \in \mathbb{R}^{d \times k}$  represent the pre-trained weight matrix of a transformer layer. QLoRA framework quantizes the full-precision weights into low-bit representation, typically using 4-bit NormalFloat (NF4), to minimize VRAM consumption.:

$$\mathbf{W}_q = Q(\mathbf{W}) \approx \mathbf{W} \quad (4-32)$$

In this expression,  $Q(\cdot)$  denotes the quantization function that maps high-precision (32-bit or 16-bit) weights into a 4-bit integer space. This process reduces the storage requirements for the backbone weights by approximately 8 times, allowing the model to remain fully loaded in memory during the fine-tuning of the adapter layers.

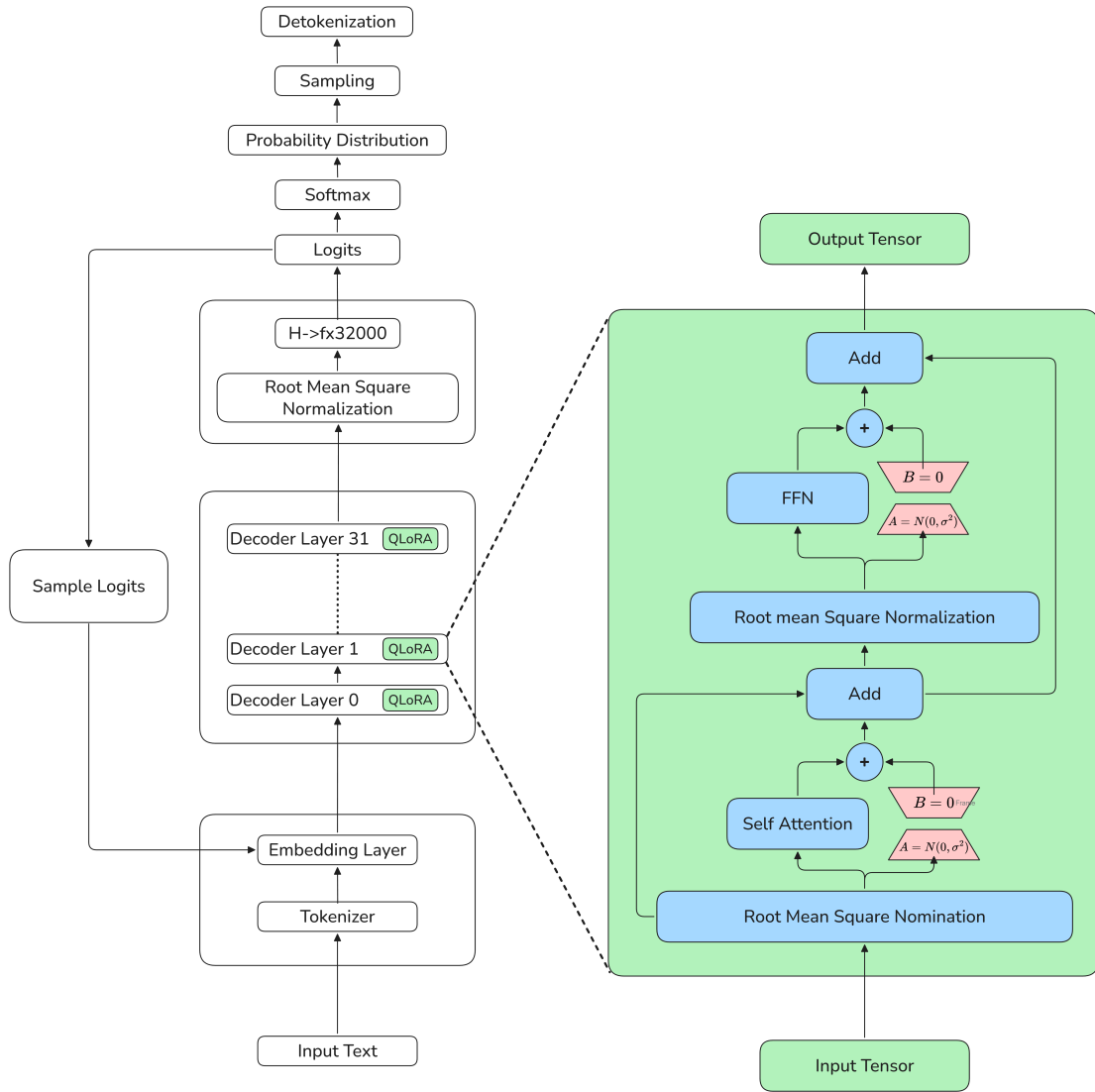


Figure 4-6 Llama Model with QLoRA

#### 4.5.2 4-bit NormalFloat (NF4) and Memory Optimization

To achieve the goal of fitting the Llama-2-7b model into approximately 5GB of VRAM, we move beyond standard linear quantization to **4-bit NormalFloat (NF4)** and **Double Quantization**.

##### 4-bit NormalFloat (NF4) Data Type

The weights of pre-trained LLMs typically follow a zero-centered normal distribution with a standard deviation  $\sigma$ . NF4 is an information-theoretically optimal data type built on *Quantile Quantization*. Instead of mapping weights to an evenly spaced grid, NF4 maps them to the quantiles of a theoretical  $N(0, 1)$  distribution. The quantization constants are calculated such that each of the 16 discrete levels in the 4-bit bin has an

equal expected number of values. For a weight  $w$ , the NF4 quantization is expressed (4-33)

$$q_i = \frac{1}{2} \left( Q_z \left( \frac{i}{2^k + 1} \right) + Q_z \left( \frac{i+1}{2^k + 1} \right) \right) \quad (4-33)$$

where  $Q_z(\cdot)$  is the cumulative distribution function (CDF) of the standard normal distribution. This ensures that the precision is highest where the weight density is greatest (near zero), significantly reducing the quantization error compared to standard 4-bit integers or floats.

### Double Quantization

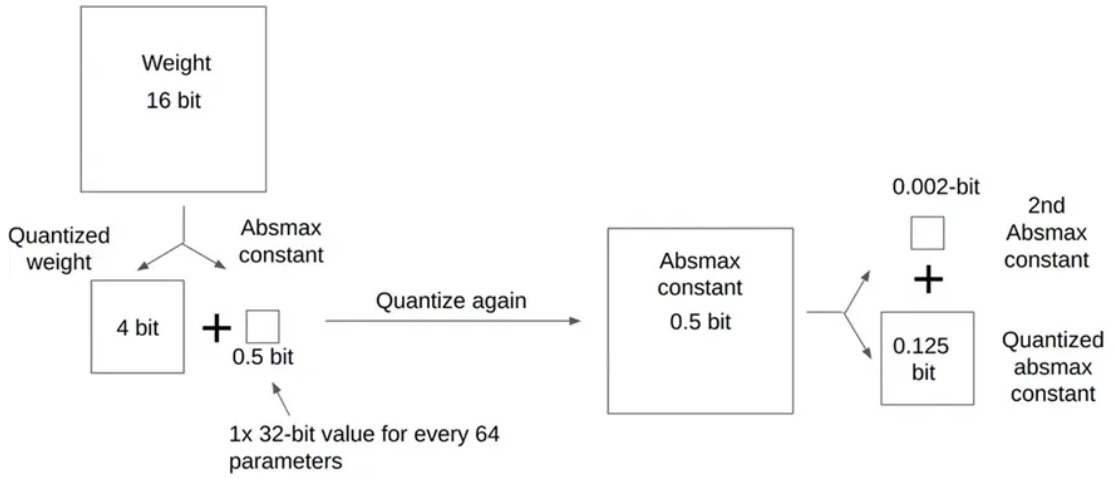


Image source: Democratizing Foundation Models via k-bit Quantization by Tim Dettmers

Figure 4-7 Double Quantization

The standard NF4 reduces the weight size but the *quantization constraints* still consume large part of VRAM. The common implementation of NF4 uses a block size of 64 meaning a 32 bit scaling factor is needed for every 64 weights, adding roughly 0,5 bits per parameter. So, **Double Quantization** treats these 32-bit constants as inputs to a *second* layer of quantization which quantizes these constants into 8-bit float (FP8) with a secondary block size of 256. The memory savings are calculated using equation (4-34)

$$\begin{aligned} \text{Memory Savings} &= \underbrace{\frac{32}{64}}_{\text{Single Quant}} - \underbrace{\left( \frac{8}{64} + \frac{32}{64 \cdot 256} \right)}_{\text{Double Quant}} \\ &\approx 0.37 \text{ bits per parameter} \end{aligned} \quad (4-34)$$

Here we can clearly observe that using double quantization reduces the overhead of quantization constants from 0.5 bits/parameter to approximately 0.127 bits/parameter.

Hence allowing the total memory footprint of the Llama-2-7b model to drop to  $\sim 4.8\text{GB}$ , well within the limits of consumer-grade hardware.

### Paged Optimizers

We utilized **Paged Optimizers** via the `bitsandbytes` library to prevent Out-of-Memory (OOM) errors during training. This mechanism utilizes NVIDIA's Unified Memory feature to map the optimizer states to the CPU RAM when the GPU's VRAM is full, and pages them back during the update step.

### Low-Rank Adaptation (LoRA)

Instead of updating the entire quantized matrix  $\mathbf{W}_q$ , which remains frozen, LoRA introduces trainable low-rank matrices  $\mathbf{A} \in \mathbb{R}^{d \times r}$  and  $\mathbf{B} \in \mathbb{R}^{r \times k}$  where the rank  $r \ll \min(d, k)$ . The adapted weight matrix is expressed as:

$$\mathbf{W}_{\text{adapted}} = \mathbf{W}_q + \Delta\mathbf{W}, \quad \Delta\mathbf{W} = \mathbf{A}\mathbf{B} \quad (4-35)$$

Here,  $\Delta\mathbf{W}$  captures task-specific updates in a low-dimensional subspace. Only  $\mathbf{A}$  and  $\mathbf{B}$  are trained, while  $\mathbf{W}_q$  remains frozen.

### Forward Pass with QLoRA

During the forward pass, the adapted weight is used in the standard linear layer to compute the output  $\mathbf{y}$  for a given input  $\mathbf{x}$ :

$$\mathbf{y} = (\text{dequant}(\mathbf{W}_q) + \mathbf{A}\mathbf{B})\mathbf{x} + \mathbf{b} \quad (4-36)$$

where  $\mathbf{b} \in \mathbb{R}^d$  represents the bias term. By combining these methods, QLoRA enables high-fidelity fine-tuning with significantly lower computational costs.

#### 4.5.3 Training Efficiency

The efficiency of QLoRA training can be summarized as follows:

- **Memory Savings:** The primary weights  $\mathbf{W}_q$  have memory savings due to storing them in 4-bit format, while only the small adapter matrices retain their full precision format.
- **Computationally Efficient:** Because  $r$  is significantly less than  $d$  or  $k$ , the number of parameters used in the training process is significantly reduced.
- **Accessible Hardware:** Training on large models is now possible using a single GPU with low VRAM.

Therefore, the key formula utilized for fine-tuning based on QLoRA in the present study is as follows:

$$\mathbf{W}_{\text{QLoRA}} = \mathbf{Q}(\mathbf{W}) + \mathbf{A}\mathbf{B} \quad (4-37)$$

## 4.6 Fine-Tuning

Fine tuning refers to the process whereby a pre-trained language model (PLM) is adapted for a particular task or domain by further training the model using a small amount of data pertaining to that particular task. The benefit of this method is that it makes use of the knowledge gained during pre-training, thus minimizing the computational resources used.

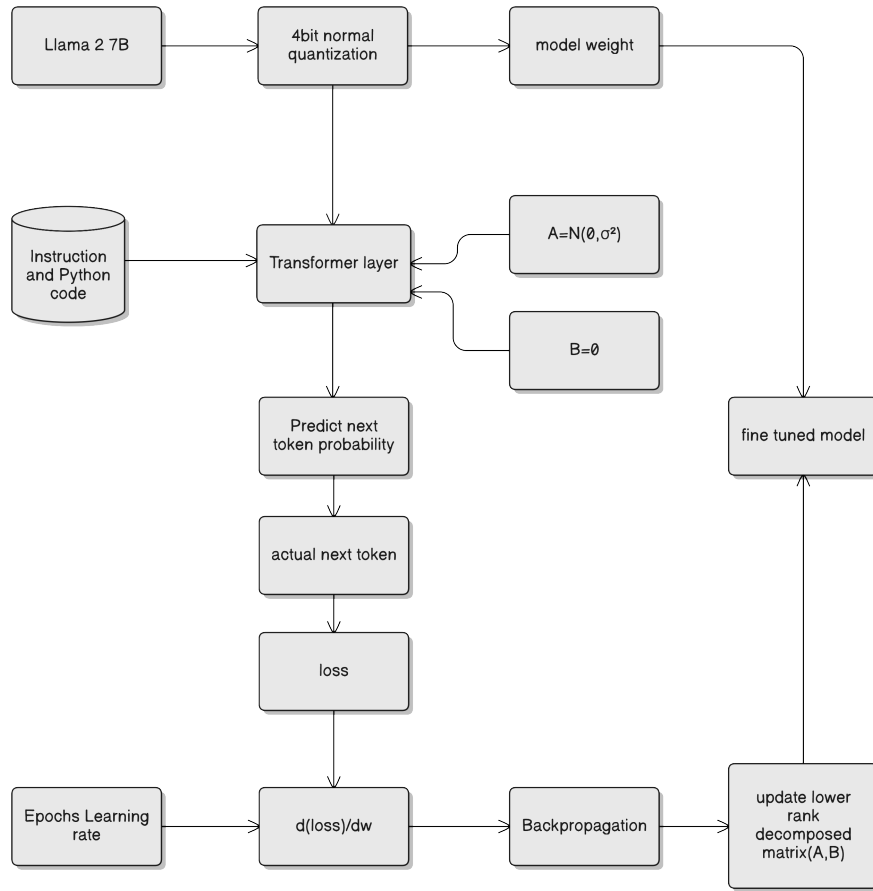


Figure 4-8 Flow Diagram of the Fine-Tuning Process

### 4.6.1 Mathematical Formulation of Fine-Tuning

Let us start by considering a model that has been pretrained using a set of parameters  $\theta \in \mathbb{R}^d$ , which is initially trained using a large general language corpus  $\mathcal{D}_{\text{pretrain}}$ . In



this case, the model defines a probability distribution of conditional probability over sequences of tokens as:

$$P_{\theta}(y \mid x)$$

where  $x$  represents the input sequence and  $y$  represents the target sequence.

For a specific downstream task characterized by a dataset  $\mathcal{D}_{\text{task}} = \{(x_i, y_i)\}_{i=1}^N$ , the fine-tuning process adjusts the parameters  $\theta$  to minimize a task-specific loss function  $\mathcal{L}(\theta)$ . This is typically achieved using the cross-entropy loss:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} \log P_{\theta}(y_{i,t} \mid y_{i,<t}, x_i) \quad (4-38)$$

Under this definition,  $y_{i,t}$  refers to the  $t - th$  element of the output sequence  $y_i$ ,  $y_{i,<t}$  represents all the elements appearing before position  $t$ , while  $T_i$  is the length of that sequence. Model parameters are optimized by a gradient descent algorithm which iterates as follows

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta) \quad (4-39)$$

where  $\eta$  represents the learning rate. Through this process, the probability distribution over model weights gets tuned to match the needs of the new target domain, without affecting the basic representations learned through pre-training.

#### 4.6.2 4-bit NormalFloat Implementation

We implemented NF4 to quantize the model so that we can achieve a memory footprint of  $\sim 5\text{GB}$  for a 7B parameter model. This method maps the weights to a specialized distribution that preserves the information density while maintaining similar performance as explained in earlier sections.

#### 4.6.3 Low-Rank Adaptation Layers

Instead of updating the full Llama-2 weight matrix  $W$ , we inject trainable rank-decomposition matrices. The update is confined to the attention projections:

$$h = W_0 x + B A x \quad (4-40)$$

where  $A$  and  $B$  are the LoRA adapters. This reduces the number of trainable parameters by over 99%.

#### 4.6.4 Instruction Masking Strategy

To ensure the model learns to generate code rather than simply repeating the prompt, we applied a custom tokenization strategy. During the preparation of the training tensors, we set the `labels` for all tokens belonging to the "Instruction" and "System" prompts to `-100`. This signals the loss function to ignore these tokens, forcing the model's weights to update based solely on the accuracy of the generated "Response" tokens.

### 4.7 Evaluation

In order to evaluate the quality of the generated code against the ground truth code, we used automatic metrics like BLEU and CodeBLEU. BLEU is an n-gram-based metric, whereas CodeBLEU takes into account factors like syntax, data flow, and semantics.

#### 4.7.1 Validation Loss

During training, we monitor the validation loss  $\mathcal{L}_{val}$ . The validation loss is computed as the average cross-entropy loss on the held-out validation set:

$$\mathcal{L}_{val} = -\frac{1}{N_{val}} \sum_{i=1}^{N_{val}} \sum_{t=1}^{T_i} \log P_{\theta}(y_{i,t} \mid y_{i,<t}, x_i) \quad (4-41)$$

A decreasing validation loss indicate that the model is learning to generate more accurate code snippets over time, while a divergence between training and validation metrics may indicate potential overfitting.

#### 4.7.2 BLEU Score

BLEU evaluates token-level n-gram overlap between generated and reference code [14]. The BLEU score is computed using Equation (4-42):

$$\text{BLEU} = \text{BP} \cdot \exp \left( \sum_{n=1}^N w_n \log p_n \right) \quad (4-42)$$

where  $p_n$  is the n-gram precision,  $w_n$  are uniform weights (typically  $w_n = 1/N$ ), and BP is the brevity penalty. BP is calculated using Equation (4-43):

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases} \quad (4-43)$$

The n-gram precision  $p_n$  uses clipped counts to avoid rewarding repeated tokens:

$$p_n = \frac{\sum_{\text{candidate sentences}} \sum_{\text{n-gram} \in \text{candidate}} \text{Count}_{\text{clip}}(\text{n-gram})}{\sum_{\text{candidate sentences}} \sum_{\text{n-gram} \in \text{candidate}} \text{Count}(\text{n-gram})} \quad (4-44)$$

### 4.7.3 CodeBLEU

CodeBLEU extends BLEU for code generation by incorporating syntax, data-flow, and semantic correctness [21]. It is defined as shown in Equation (4-45):

$$\text{CodeBLEU} = \alpha \cdot \text{BLEU} + \beta \cdot \text{Match}_{\text{AST}} + \gamma \cdot \text{Match}_{\text{DF}} + \delta \cdot \text{BLEU}_{\text{weighted}} \quad (4-45)$$

where  $\alpha$ ,  $\beta$ ,  $\gamma$ , and  $\delta$  are weighting parameters that control the contribution of each component.

The individual components are computed as follows:

AST similarity is calculated by comparing the abstract syntax trees of the generated and reference code, measuring the proportion of matching subtrees:

$$\text{Match}_{\text{AST}} = \frac{\text{matching\_subtrees}}{\text{total\_subtrees}} \quad (4-46)$$

DFScore evaluates the correctness of data flow by comparing the data flow graphs of the generated and reference code, measuring the proportion of matching data flow edges:

$$\text{Match}_{\text{DF}} = \frac{\text{matching\_dataflow\_edges}}{\text{total\_dataflow\_edges}} \quad (4-47)$$

The weighted BLEU component is computed using syntax-aware token weights so that language keywords and structurally important tokens contribute more than common tokens:

$$\text{BLEU}_{\text{weighted}} = \frac{\sum_{g \in \mathcal{G}} w(g) \text{Count}_{\text{clip}}(g)}{\sum_{g \in \mathcal{G}} w(g) \text{Count}(g)} \quad (4-48)$$

where  $\mathcal{G}$  denotes the set of n-grams and  $w(g)$  is the predefined weight for n-gram  $g$  based on token importance.

### 4.7.4 Functional Evaluation

We test our model on human-eval questions, which consist of a problem statement, a reference solution, and a set of unit tests. The generated code is executed against these unit tests to verify its correctness. We evaluate for pass@k, which measures the percentage of problems for which at least one of the top-k generated solutions passes all

unit tests. This metric directly assesses the functional correctness of the generated code. The human-eval score was found using the following formula:

$$\text{pass@k} = 1 - \left(1 - \frac{c}{n}\right)^k \quad (4-49)$$

Where  $c$  is the number of correct solutions among  $n$  total generated samples. By evaluating at multiple  $k$  values, we can assess both the reliability of the model (pass@1) and its best-effort performance (pass@5).

### OpenAI HumanEval Dataset

The evaluation of logical reasoning skills and syntactic correctness of the fine-tuned model is carried out using the benchmark OpenAI HumanEval dataset [22]. The selection of this dataset is justified by the fact that it is considered to be the state-of-the-art one for measuring zero-shot code generation quality for Large Language Models, paying attention more to functional correctness than to text similarity.

HumanEval consists of 164 tasks which are manually designed Python programming tasks. It includes triplets where:

- **Task ID:** the unique identifier of the coding task.
- **Prompt:** contains a function signature, natural language description with some example test cases (assertions).
- **Canonical Solution:** a reference Python implementation of the task.
- **Test Suite:** An extensive suite of unit tests that remain invisible but on average include 7.7 tests for each problem. The solution to the problem can be considered valid only if all these tests are passed.

The dataset comprises various problems that require such skills as string manipulations and mathematical calculations as well as those requiring algorithm knowledge to complete. Since they were written by humans, these problems would most probably not occur in the original dataset utilized during pre-training, like GitHub and Stack Overflow. Hence, there is no possibility of data leakage here.

### Pass@k Metric and Unbiased Estimation

Code generation is a stochastic process so simple success which measures if a model can solve a problem in single attempt cannot be used. We therefore utilize the **pass@k** metric. We have used the unbiased estimator defined by OpenAI to account for high variance in small sample sizes:

$$\text{pass@k} = \mathbb{E}_{\text{Problems}} \left[ 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \quad (4-50)$$

Where:

- $n$  is the total number of independent samples generated per problem (we use  $n = 5$ ).
- $c$  is the number of those samples that successfully passed the unit tests.
- $k$  is the specific rank being evaluated ( $k \in \{1, 2, 3, 4, 5\}$ ).

We evaluate the model at multiple  $k$  values as lower  $k = 1$  values help determine the model's reliability and higher  $k = 5$  value help determine the model's best solution, which represents the likelihood of a human user finding a correct solution within five generated suggestions. The syntactical, logical and semantic meaningfulness of the generated code is correctly verified by combining execution based testing and similarity metrics.

#### 4.8 Ollama for Local Inference

We implemented our model to run on Ollama [23] for local inference, which is a lightweight framework designed to run large language models efficiently on consumer-grade hardware. We strictly avoided use of cloud-based APIs and implemented Ollama based approach as the scope of our project was to run the model on consumer grade hardware in resource constrained environment. The fine-tuned llama-2-7b model is integrated into the Ollama environment to support downstream tasks such as code generation. This setup provides several advantages, especially when used in scenarios that require timely responses and when limited connectivity exists within the network. In addition, having a local system ensures greater security and privacy of data since the input data is processed completely in the user's machine without uploading anything to remote machines. Therefore, this system would be appropriate for use in cases involving private codes and data. Within the larger pipeline, Ollama acts as the inference engine. After training the models, the parameters are uploaded to the Ollama inference engine and queried using prompts.

##### 4.8.1 Functional and Non-Functional Requirements

The use of Ollama for local inference fulfills the requirements of the system both functionally and non-functionally.

##### Functional Requirements

- The system shall produce code based on the user prompt.

- The system shall allow the use of fine-tuned llama-2-7b inference.
- The system shall facilitate prompt-based interaction to handle queries dynamically.

Ollama fulfills the above-listed functional requirements as it enables the seamless integration and execution of the fine-tuned model to process user inputs and produce appropriate outputs.

### **Non-Functional Requirements**

- *Performance:* Low latency in inference is enabled by the use of on-device GPU power.
- *Privacy:* The data is processed locally and no data are sent to other systems.
- *Reliability:* The system does not require internet connectivity
- *Scalability:* The system can be used across different local environments without the use of any cloud.

By leveraging Ollama, the system effectively meets these non-functional requirements, making it suitable for real-world applications where efficiency, privacy, and reliability are critical.

## 5 IMPLEMENTATION DETAILS

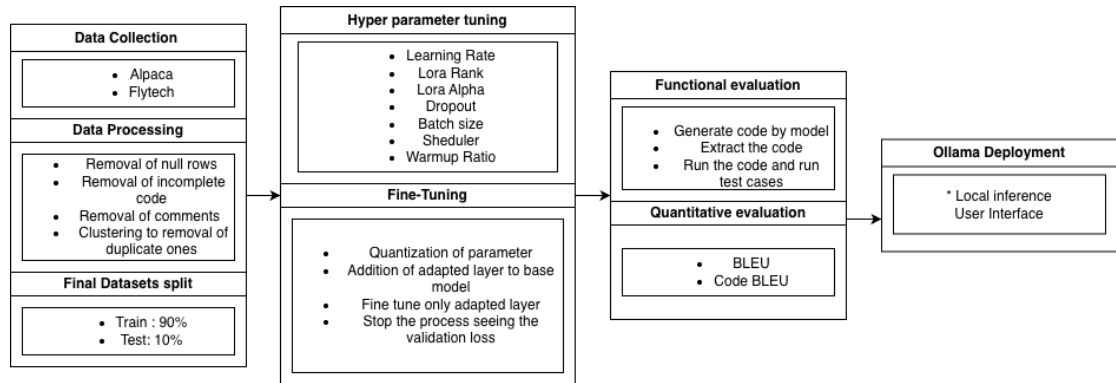


Figure 5-1 Implementation of the Project

### 5.1 Data Collection and Preparation

A high-quality and task-oriented dataset that included different programming tasks along with their solutions was needed for fine-tuning Llama-2 7B for Python code generation. We created a dataset by combining data from several reliable open-source datasets.

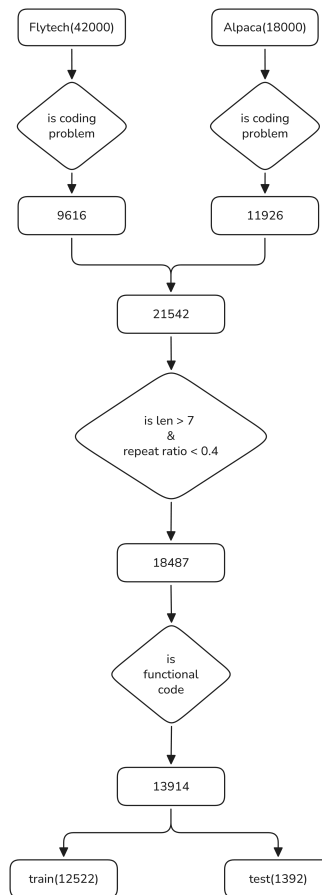


Figure 5-2 Dataset Preparation Pipeline

Table 5-1 Datasets Used for Fine-tuning Llama-2 7B for Python Code Generation.

| Dataset                                          | Size          | Description                                                                                                                                                                     | License      |
|--------------------------------------------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| FlyTech Python Code Dataset (2025) [19]          | ~42,000 pairs | Python scripts with comments, docstrings, and structured tasks. Curated for code generation with high-quality, well-commented code and diverse algorithmic/real-world examples. | MIT License  |
| Python Code Instructions 18k (Alpaca-style) [20] | ~18,000 pairs | Instruction prompts and Python responses. Matches instruction-tuning structure; provides human-readable prompts and increases prompt diversity.                                 | CC BY-NC 4.0 |

### 5.1.1 Dataset Sources

### 5.1.2 Dataset Preparation and Refinement

### 5.1.3 Alpaca Dataset Refinement

The main source behind the instruction-following training data was the **iamtarun/python\_code\_instructions\_18k\_alpaca** source code repository. For refining the model towards logical reasoning and algorithmic programming, the below process was adopted:

- **Automated Classification:** The qwen2.5-coder:1.5b model was used for analyzing the 18,000 entries, and was directed to select those problems which included algorithmic reasoning and programming competitions.
- **Filtering Results:** In total, 12,000 problems were selected based on their quality. Following that, a more thorough manual filtering was done to remove any wrongly classified entries, for instance, general coding tasks rather than reasoning problems.
- **Final Distribution:** Upon filtering of false-positive rows and removal of duplicates, the final refined Alpaca dataset contained 11,926 examples of Python code described by instructions.

### 5.1.4 FlyTech Dataset Processing

To supplement the algorithmic data with wider technical coding patterns, the FlyTech Python dataset was used. Since the dataset had a large number of entries initially, it was crucial to develop a comprehensive cleaning strategy:



Datasets: lamtarun/python\_code\_instructions\_18k\_alpaca like 330 Dataset card Data Studio Files and versions xet Community

Split (1)  
train · 18.6k rows

Search this dataset

| instruction<br>string · lengths                                            | input<br>string · lengths                                                                | output<br>string · lengths                                                                             | prompt<br>string · lengths                                                                          |
|----------------------------------------------------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| 21<br>7.76k                                                                | 0<br>880                                                                                 | 1<br>20.8k                                                                                             | 212<br>21.4k                                                                                        |
| Create a function to calculate the sum of a sequence of integers.          | [1, 2, 3, 4, 5]                                                                          | # Python code def sum_sequence(sequence): sum = 0 for num in sequence: sum += num return sum           | Below is an instruction that describes a task. Write a response that appropriately completes the... |
| Generate a Python code for crawling a website for a specific type of...    | website: www.example.com data to crawl: phone numbers                                    | import requests import re def crawl_website_for_phone_numbers(website): respons...                     | Below is an instruction that describes a task. Write a response that appropriately completes the... |
| Create a Python list comprehension to get the squared values of a list...  |                                                                                          | [x*x for x in [1, 2, 3, 5, 8, 13]]                                                                     | Below is an instruction that describes a task. Write a response that appropriately completes the... |
| Generate a python script to perform this action.                           | Given a string, remove all the consecutive duplicates from the string. Input: "AABBCCCD" | def remove_duplicates(string): result = "" prev = '' for char in string: if char != prev: result +=... | Below is an instruction that describes a task. Write a response that appropriately completes the... |
| Write a python script to generates random numbers between 0 and 9 that...  |                                                                                          | def generate_random_divisible_number(): import random while True: # Generate a random number...        | Below is an instruction that describes a task. Write a response that appropriately completes the... |
| Write a Python code to get the third largest element in a given row.       | [12, 13, 13, 45, 22, 99]                                                                 | def third_largest(lst): if len(lst) < 3: return distinct = [] for i in lst: if i not in distinct:...   | Below is an instruction that describes a task. Write a response that appropriately completes the... |
| Generate a REST API with Python and Flask that allows users to create,...  | Not applicable                                                                           | from flask import Flask, request from flask_sqlalchemy import SQLAlchemy app =...                      | Below is an instruction that describes a task. Write a response that appropriately completes the... |
| Create a Python function that takes in a string and a list of words and... | 'This is a test', ['test', 'this', 'is']                                                 | def contains_words(input_string, words): for word in words: if word not in input_string: return...     | Below is an instruction that describes a task. Write a response that appropriately completes the... |

< Previous 1 2 3 ... 187 Next >

Figure 5-3 Alpaca Datasets

- **First Cleaning:** Starting with 42,000 entries, the first step consisted in developing code that would detect and filter out duplicate and almost duplicate samples. Furthermore, the problem descriptions that had null values or missing entries were removed to prevent the model from being trained on incomplete data.
- **Second Cleaning:** After the first step of cleaning that narrowed down the data pool to 22,000, another round of filtration based on heuristics was conducted to identify the best samples.
- **Final Samples:** The result of this process was 9,616 samples of the highest quality

Datasets: flytech/python-codes-25k like 167 Dataset card Data Studio Files and versions xet Community

Split (1)  
train · 49.6k rows

Search this dataset

| output<br>string · lengths                                                                                | instruction<br>string · lengths                       | input<br>string · lengths                                 | text<br>string · lengths                                                                                  |
|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 1<br>2.06k                                                                                                | 4<br>1.71k                                            | 0<br>163                                                  | 57<br>2.4k                                                                                                |
| '''python tasks = [] while True: task = input('Enter a task or type 'done' to finish: ') i...             | Help me set up my daily to-do list!                   | Setting up your daily to-do list...                       | Help me set up my daily to-do list! Setting up your daily to-do list... '''python tasks = [] while...     |
| '''python shopping_list = {} while True: item = input('Enter an item or type 'done' to finish: ')...      | Create a shopping list based on my inputs!            | Creating a shopping list...                               | Create a shopping list based on my inputs! Creating a shopping list... '''python shopping_list = {}...    |
| '''python total_time = 0 for i in range(1, 8): time = float(input(f'Enter phone usage in hours for day... | Calculate how much time I spend on my phone per week! | Calculating weekly phone usage...                         | Calculate how much time I spend on my phone per week! Calculating weekly phone usage... '''python...      |
| '''python total_bill = float(input('Enter the total bill amount: ')) friends_count = int(input('Enter...  | Help me split the bill among my friends!              | Splitting the bill...                                     | Help me split the bill among my friends! Splitting the bill... '''python total_bill =...                  |
| '''python movie_list = {} while True: genre = input('Enter a movie genre or type 'done' to...             | Organize my movie list into genres!                   | Organizing your movie list...                             | Organize my movie list into genres! Organizing your movie list... '''python movie_list = {} while True... |
| '''python ratings = [] while True: rating = input('Enter a book rating or type 'done' to...               | Calculate the average rating of my book collection!   | Calculating the average rating of your book collection... | Calculate the average rating of my book collection! Calculating the average rating of your book...        |
| '''python mood = input('What's your mood today? ') playlist = [] while True: song = input(f'Enter a...    | Create a playlist based on my mood!                   | Creating a playlist...                                    | Create a playlist based on my mood! Creating a playlist... '''python mood = input('What's your...         |
| '''python best_deals = {} while True: item = input('Enter a shopping item or type 'done' to...            | Help me find the best deals on my shopping list!      | Finding the best deals...                                 | Help me find the best deals on my shopping list! Finding the best deals... '''python best_deals = {...    |

< Previous 1 2 3 ... 497 Next >

Figure 5-4 FlyTech Datasets

### 5.1.5 Consolidated Training Set

Through the combination of the improved Alpaca and FlyTech data sets, a final data set optimized for generating Python codes was created. This combination ensures that there is an equilibrium between programming logic and software engineering tasks.

Table 5-2 Summary of Dataset Refinement Metrics

| Dataset Source    | Raw Count     | After Cleaning | Final Selected |
|-------------------|---------------|----------------|----------------|
| Alpaca (iamtarun) | 18,000        | ~12,000        | <b>11,926</b>  |
| FlyTech Python    | 42,000        | 22,000         | <b>9,616</b>   |
| <b>Total</b>      | <b>60,000</b> | <b>34,000</b>  | <b>21,542</b>  |

### 5.1.6 Data Filtering and Complexity Heuristics

To avoid degradation of the fine-tuning process from low entropy or repeated data, a specially programmed Complexity Filtering Layer was introduced. The programming layer focused on addressing the following two forms of noise sources: trivial or uninformative solutions and the common loop-hallucinations pattern found in web-scraper data.

#### Length-Based Thresholding

Tiny code snippets like a single variable assignment or an empty function definition do not provide sufficient structural context for a 7B parameter model to learn complex logic. In order to ensure that every sample must have at least one function, we enforced a lower limit on the number of tokens per sample ( $\tau_{min} = 7$ ). The choice of seven tokens as the default is based on the structure of a basic function in Python. A basic function snippet, such as `def func(a, b): return a + b` consists of seven unique tokens. Through this requirement, we ensured that the model had to learn from dense logic snippets instead of trivial samples, which do not contribute much to training.

#### Lexical Diversity and Repetition Analysis

It is observed that the primary failure modes of Large Language Models (LLMs) is the generation of repetitive, circular code blocks. To prevent the model from capturing these patterns during fine-tuning, we calculated a Repetition Ratio () for each candidate sample:

$$\rho = 1.0 - \left( \frac{|\text{Unique Tokens}|}{|\text{Total Tokens}|} \right) \quad (5-1)$$

The selection of the optimal threshold was determined empirically by evaluating the relationship between repetition ratio and resulting dataset size to balance data volume with quality, as summarized in 5-3.

Table 5-3 Impact of Repetition Ratio Threshold on Dataset Volume

| Repetition Ratio ( $\rho$ ) | Final Dataset Size (Samples) |
|-----------------------------|------------------------------|
| 0.7                         | 20,335                       |
| 0.6                         | 20,270                       |
| 0.5                         | 19,876                       |
| <b>0.4</b>                  | <b>18,487</b>                |
| 0.3                         | 15,577                       |

Table 5-4 Data Filtering Heuristics and Constraints Summary

| Constraint            | Threshold  | Rationale                                                                             |
|-----------------------|------------|---------------------------------------------------------------------------------------|
| Min. Word Count       | > 7 tokens | Ensures functional complexity and presence of a return block.                         |
| Max. Repetition Ratio | < 0.4      | Filters out circular logic and redundant boilerplate; selected via empirical testing. |

This filtering process was necessary for stabilizing the training trajectory and reducing over fitting on trivial examples. On prioritizing unique algorithmic transformations over repetitive syntactic structures, the model demonstrated significantly better generalization to unseen tasks. Based on this experimentation, we enforced a maximum threshold of < 0.4. This heuristic identifies and removes samples where more than 40% of the content is redundant. By pruning these samples, we effectively filtered out “boilerplate” code and broken generations while preserving high-quality, logically dense algorithmic solutions.

### 5.1.7 Removal of Comments and Doc-strings

To ensure the model captures only the functional logic not the natural language explanations, a cleaning phase was implemented to filter-out comments and doc-strings from the Python source code from the final dataset. This process extracts pure algorithmic structure only, preventing the model from over-relying on internal documentation in the solution part which may not always align with the executed logic.

Table 5-5 Dataset Distillation and Logic Density Improvement

| Stage                      | Sample Count | Est. Logic Density |
|----------------------------|--------------|--------------------|
| Raw Combined               | 60,000       | Low                |
| Filtered (Logic Only)      | 34,000       | Medium             |
| Refined (Functional/Clean) | 21,542       | High               |
| Final (Functional-Only)    | 13,914       | Very High          |

## Regex-Based Feature Isolation

To remove the non-functional code we used regular expression:

1. **Single-Line Comments:** Identified via the pattern `#.*?\n`, ensuring that inline annotations are removed without disrupting the line-break structure.
2. **Triple-Quote Docstrings:** Both double-quote (`"""..."""`) and single-quote (`'''...'''`) docstrings were targeted using the `DOTALL` flag to remove multi-line string literal documentation.

The transformation process can be represented by the mapping function  $f(C) \rightarrow C_{pure}$ , where  $C$  is the original source code. By removing these elements, we reduce the token count per sample while increasing the "logic density". This ensures that the attention mechanism of the Llama-2-7B model is focused strictly on variable transformations, control flow, and library calls.

Table 5-6 Preprocessing Impact of Docstring Removal

| Element Removed     | Pattern Strategy                 | Objective                             |
|---------------------|----------------------------------|---------------------------------------|
| Inline Comments     | <code>re.sub(r'#[.*?]\n')</code> | Remove human annotations              |
| Block Docstrings    | <code>re.DOTALL</code> Regex     | Eliminate non-executable descriptions |
| Whitespace Trimming | <code>strip()</code>             | Maintain clean indentation blocks     |

## Impact on Training Efficiency

Removing these components resulted in a more compact representation of the dataset. As shown in the training logs, models trained on "clean" code converged faster because the loss function was calculated on functionally relevant tokens only. This step effectively reduced the noise-to-signal ratio, allowing the fine-tuned adapters to capture the patterns of Python's syntax more effectively.

### 5.1.8 Train and test Split

For evaluation of the model, the dataset was partitioned into a 90% training set and a 10% test set, ensuring that the distribution of problem types and difficulty levels was preserved across both subsets. This split was crucial for evaluating the model's generalization capabilities, particularly on unseen problem types that may not have been presented in the training data.

### 5.1.9 Llama-2 Chat Template and Tokenization Mechanics

To improve the pre-trained conversational abilities of the Llama-2-7B-Chat model, all dataset samples were mapped to the specific Llama-2 Chat Template format. This

formatting is essential for the model to correctly identify the start and end of a user request versus the assistant’s generation.

```
<s>[INST]
{user_instruction} [/INST] {code_solution} </s>
```

[illegible]

## 5.2 Hyperparameter Optimization

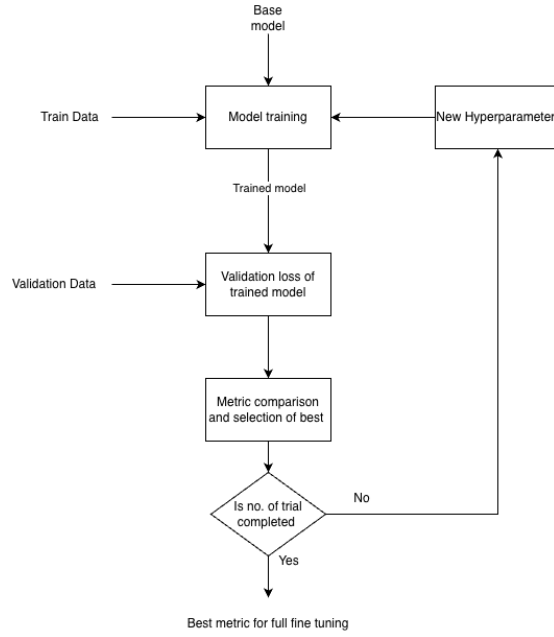


Figure 5-6 Hyperparameter Optimization with Random Search

### 5.2.1 Bayesian Optimization via TPE

Instead of using the Grid Search method or random sampling by Random Search, the **Tree-structured Parzen Estimator (TPE)** [24]. The latter is a Bayesian method of optimization where

1. **Exploration:** In initial trials (0-10), the sampler populates the search space.
2. **Exploitation:** For subsequent trials, the TPE algorithm constructs a probability model of the hyperparameter performance, favoring regions that historically yielded a lower validation loss.

Moreover, a Median pruner was applied in order to make calculations more efficient since it controls the intermediate loss and stops trials which cannot surpass the “best” trial according to statistical criteria.

### 5.2.2 Search Space Rationale

The search space was carefully chosen to fit the unique requirements of the Llama-2-7b architecture and the code generation dataset:

- **Learning Rate ( $\eta$ ):** We targeted a narrow log-space ( $8 \times 10^{-6}$  to  $3 \times 10^{-5}$ ) as high learning rates can cause catastrophic forgetting or gradient explosion.

- **LoRA Configuration:** While a higher  $r$  (e.g., 32) allows for more complex parameter updates, but for code generation tasks, it is observed that a lower rank with a high  $\alpha$  (scaling factor) provides better regularization.
- **Scheduler Topology:** We compared linear decay against cosine and cosine\_with\_restarts. The cosine scheduler is used because it maintains a higher learning rate for longer, allowing the model to escape local minima before gracefully decaying.

### 5.2.3 Quantized Training Implementation

In order to train on consumer hardware without compromising performance, we employed **Quantized Low-Rank Adaptation (QLoRA)**. The following is the configuration of QLoRA.

- **4-bit NormalFloat (NF4):** This is the ideal choice for weights that follow the normal distribution pattern, which is seen in pre-trained LLMs.
- **Double Quantization:** In quantizing quantization constants, we were able to save an average of 0.37 bits per parameter.
- **Paged Optimizers:** In order to prevent Out-of-Memory (OOM) issues due to increases in sequence length, we used NVIDIA’s unified memory.

### 5.2.4 Technical Rationale for Search Space Boundaries

Our choice of search space for Optuna wasn’t random but rather driven by the necessities of the **Low-Rank Adaptation (LoRA)** process.

- **Rank ( $r$ ) and Alpha ( $\alpha$ ):** We tested ranks {8,16,32}. While higher ranks increase the degrees of freedom, they also increase the risk of overfitting on specific coding styles. We prioritized the scaling factor  $\alpha = 4 \times r$ , based on the observation that a larger alpha helps the model "assert" its new knowledge over the frozen base weights.
- **The Dropout Factor:** We included a dropout range of [0.05,0.15]. In code generation, dropout acts as a critical regularizer, preventing the model from memorizing specific variable names in the training set and instead forcing it to learn generalized Python syntax.

### 5.2.5 SFT Loss Masking

To ensure the model learned the logic of the code rather than the structure of the prompt, we applied Instruction Masking. During the calculation of the Cross-Entropy loss, all tokens corresponding to the user’s “prompt” were assigned a label of 100, so the gradient

updates were applied to the “code” part, focusing the model’s capacity on generating syntactically and logically correct solution.

### 5.2.6 Formalism of Instruction Masking

In standard Supervised Fine-Tuning (SFT), the goal is to minimize the negative log-likelihood of the target sequence. However, in our implementation, we modified the objective function to focus strictly on code synthesis. Given a sequence of tokens  $Y = \{y_1, y_2, \dots, y_n\}$ , where  $Y_{prompt} \subset Y$  and  $Y_{code} \subset Y$ , the loss  $\mathcal{L}$  is calculated as:

$$\mathcal{L} = - \sum_{i \in Y_{code}} \log P(y_i | y_{<i})$$

By assigning  $y_i = -100$  for all  $i \in Y_{prompt}$ , the indices are ignored by the PyTorch `CrossEntropyLoss` function. This "Completion-only" training ensures that the model does not waste representational capacity on the user’s input. This is particularly important for Python generation, as the model must learn that the transition from the `[/INST]` tag to the `def` keyword.

## 5.3 Fine-Tuning Implementation

After the identification of optimal hyperparameter set through Bayesian optimization, the final fine-tuning phase was executed. This process utilized the **QLoRA (Quantized Low-Rank Adaptation)** methodology to adapt the `Llama-2-7b-chat-hf` model to the specialized domain for competitive programming using python as a programming language.

### 5.3.1 Instruction Masking and Data Engineering

An important component of the approach involves the utilization of Instruction Masking (completion-only training). In traditional causal language modeling, the model predicts every token of the text. Nonetheless, when training a coding assistant, predicting the input prompt itself is unnecessary and distracting.

- **Boundary Detection:** The `preprocess_function_with_masking` identifies the precise token boundary between the user’s prompt and the assistant’s code completion.
- **Loss Suppression:** By following the PyTorch label masking approach, prompt tokens receive an index of -100, and those indices do not contribute towards the calculation of the Cross-Entropy loss.
- **Optimization Focus:** This process guarantees that the gradient updates during training are exclusively focused on the syntax and semantic validity of the code



being generated by the model, allowing the network to use its full computational potential for problem-solving purposes only.

### 5.3.2 Architectural Efficiency: QLoRA and NF4

For optimizing our hardware capabilities in order to reach optimal performance levels, **4-bit NormalFloat (NF4)** was added.

- **Quantization Strategy:** NF4 is considered the most optimal when it comes to normal weight distributions. Through double quantization, we decreased the memory requirement even more through quantization of quantization constants.
- **Comprehensive Adaptation:** Based on the HPO results, LoRA adapters were applied to all linear layers, including the MLP blocks (`gate_proj`, `up_proj`, `down_proj`). This allows the model to capture deeper structural patterns in complex programming languages.

### 5.3.3 Optimization and Training Dynamics

The last training session was conducted using a Cosine Learning Rate Scheduler that offers a smooth decay after a warmup phase.

- **Warmup Strategy:** The warmup ratio was chosen to be 6.93%, allowing the model to reach stability for its 4-bit weights and adapter gradients before it reaches its peak learning rate of  $2.9110^5$ .
- **Effective Batch Size:** To ensure the gradient descent is stable, we used a batch size of 4 per device and set Gradient Accumulation to 4, giving us an effective batch size of 16.
- **Memory Management:** For optimal utilization of memory resources, `paged_adamw_32bit` and Gradient Check-pointing were used. This allowed us to transfer optimizer states to the CPU during VRAM spikes and recomputed the forward pass during back-propagation steps.

### 5.3.4 Configuration Summary

The final model configuration used for fine-tuning the LLaMA-2-7B-Chat model is summarized in Table 5-7. The configuration includes the base model, quantization settings, LoRA hyperparameters, optimizer, scheduler, and other important training parameters. The training process was monitored using Weights & Biases (WandB) and the final model was pushed to the Hugging Face Hub for cloud-based repository.

Table 5-7 Final Fine-Tuning Hyperparameters and Training Configuration

| Feature                     | Implementation Detail                                            |
|-----------------------------|------------------------------------------------------------------|
| Base Model                  | Llama-2-7b-chat-hf                                               |
| Fine-Tuning Method          | QLoRA                                                            |
| Quantization                | 4-bit NF4 with Double Quantization                               |
| Maximum Sequence Length     | 512                                                              |
| Batch Size (per device)     | 4                                                                |
| Gradient Accumulation Steps | 4                                                                |
| Learning Rate               | $2.9 \times 10^{-5}$                                             |
| Warmup Ratio                | 0.0693                                                           |
| Learning Rate Scheduler     | Cosine Decay                                                     |
| Optimizer                   | Paged AdamW 32-bit                                               |
| Gradient Checkpointing      | Enabled                                                          |
| Precision                   | FP16 Mixed Precision                                             |
| Random Seed                 | 42                                                               |
| LoRA Rank ( $r$ )           | 8                                                                |
| LoRA Alpha ( $\alpha$ )     | 32                                                               |
| LoRA Dropout                | 0.0961                                                           |
| LoRA Target Modules         | q_proj, k_proj, v_proj, o_proj,<br>gate_proj, up_proj, down_proj |
| Loss Strategy               | Instruction Masking (Labels = -100)                              |
| Evaluation Strategy         | Step-based evaluation every 50 steps                             |
| Model Logging               | Weights & Biases                                                 |
| Model Repository            | Hugging Face Hub                                                 |

### 5.3.5 Resources for Fine-Tuning

The fine-tuning of the LLaMA-2-7B model was conducted on a Kaggle environment equipped with dual NVIDIA T4 GPUs. The GPUs setup provided sufficient memory and compute capability to handle large language models in our case. The training process lasts for approximately 3 hours and 2 minutes.

During fine-tuning, the training process was intentionally stopped at nearly epoch 2. This decision was based on monitoring the validation loss, as by epoch 2, the validation loss had plateaued, indicating that on further training the model would start overfitting.

To optimize memory usage and computational efficiency, we configured 4-bit quantization using the BitsAndBytes library. And, Parameter-Efficient Fine-Tuning (PEFT) techniques were used that reduced the number of trainable parameter significantly. The combination of these approaches allowed the large model to be trained within the Kaggle GPU.

Overall, a large-scale models such as LLaMA-2 can be fine-tuned efficiently using accessible cloud resources.

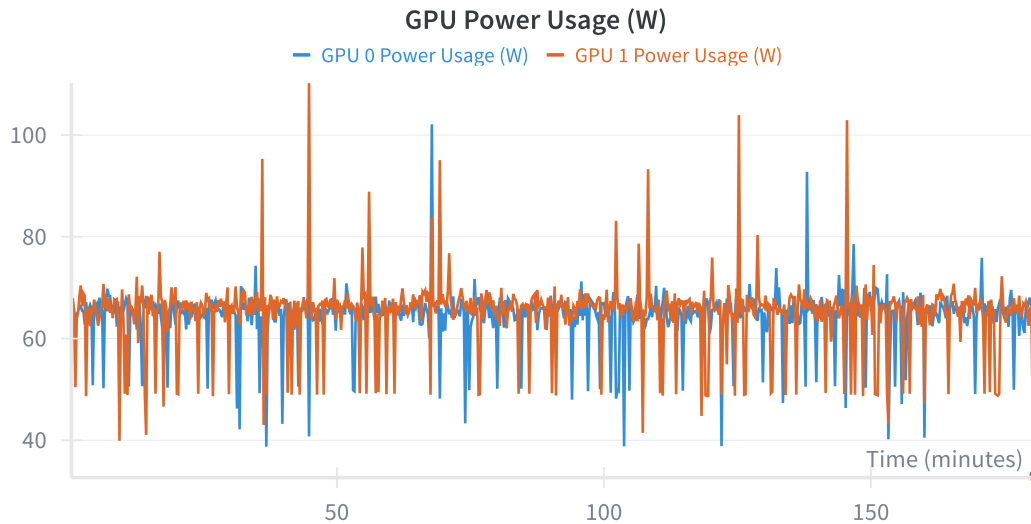


Figure 5-7 GPU Usage During the Fine-Tuning Process, Showing Utilization Trends Across the Dual T4 GPUs.

Overall, this setup demonstrates that large-scale models such as LLaMA-2 can be fine-tuned efficiently using accessible cloud resources, parameter-efficient techniques, and careful monitoring of training metrics.

## 5.4 Evaluation

For the evaluation of the fine-tuned Llama-2-7b model, we used a multi-layered software ecosystem designed to ensure reproducibility and rigorous performance tracking.

### 5.4.1 Software Infrastructure and Integration

The evaluation pipeline employed a number of advanced software packages which created an integrated workflow ranging from experiment design to post-training model validation:

- **Weights & Biases (W&B):** The WB toolset was implemented for dynamic visualization of the training process. It allowed tracking the Validation Loss and Gradient Norms in 50-step increments to ensure that the network did not exhibit unstable gradient behavior.
- **Hugging Face Hub API:** The `huggingface_hub` package was used to implement a version control system. Every checkpoint was uploaded to a dedicated repository. With the help of the software integration, "checkpointed evaluation" was performed, meaning that the intermediate network states were validated in relation to the fully-converged weights.

- **PEFT and BitsAndBytes:** The PEFT library was used to fine-tune the llama-2 7B model for the python code generation task. Only a 160 MB adapter file was injected into the frozen 4-bit base model.

#### 5.4.2 Standardized Benchmarking: OpenAI HumanEval

To gauge the coding ability of the fine-tuned model, we utilized the OpenAI HumanEval dataset. This dataset contains 164 hand-crafted programming tasks meant to assess logic and algorithms. We implemented the Pass@k metric, which measures the probability that at least one of the generated code samples passes the tests for a given problem. The software computes this using the unbiased estimator with this formula:

$$\text{pass}@k = \mathbb{E} \left[ 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$

where  $n$  is the total number of samples generated per problem (set to  $n = 5$  in our implementation), and  $c$  is the number of samples that successfully passed the unit tests.

#### 5.4.3 Qualitative Assessment and Code Extraction

We implemented a specialized inference and extraction pipeline (`evaluation.py`) to evaluate the practical usability of the model outputs. Because LLMs often wrap code in conversational “chatter,” software identifies code blocks between triple backticks (`"`python ... "``) and falls back to keyword-based detection (`def`, `class`, `import`) to ensure a clean executable string is retrieved for the `check_correctness` function before passing to the model for training.

#### 5.4.4 Sampling Strategies for HumanEval

The performance on the HumanEval benchmark heavily depends on the decoding strategies. So, we implemented a **Stochastic Decoding** pipeline to maximize the Pass@k scores.

- **Temperature Scaling** ( $T = 0.8$ ): This softens the probability distribution, allowing the model to generate diverse algorithmic solutions across its five attempts per problem, allowing higher degree of freedom.
- **Top-p (Nucleus) Sampling** ( $p = 0.95$ ): This restricts the model’s vocabulary to the smallest set of tokens to be generated whose cumulative probability exceeds 0.95. This prevents hallucinations for creative tasks to solve complex problems.

#### 5.4.5 Evaluation between quantized and unquantized base models

To ensure that there was no significant impact on the model's accuracy and efficiency due to quantization process, a comparison between the Base Llama-2-7b-chat (not quantized) and the Base Llama-2-7b-chat (quantized) was done. In the comparison, both models were tested on the same subset of 1392 test case problems using similar sampling parameters (temperature=0.7,  $top_p = 0.9$ ). The testing procedure included coding of each test case in the subset and obtaining BLEU and CodeBLEU scores with respect to the reference code. Both model had similar evaluation metric which means that the quantization did not have any significant negative effect on the models capacity of generating and reasoning Python code. In CodeBleu, the generated codes were evaluated based on four criteria namely N-gram Matching, Weighted N-gram Matching, Syntax Matching, and Data Flow Matching. Based on the results obtained, it can be seen that quantized base model had relatively similar scores with the unquantized base model

#### 5.4.6 Quantitative Metrics of base and fine-tuned model

Post refinement, the base model and the fine-tuned model were tested on the same dataset of 1392 test cases. The testing was done by first coding for each test case and finding BLEU score and CodeBLEU score for the coded program compared to the reference code. It was seen that there was a considerable improvement in the BLEU and CodeBLEU scores of the fine-tuned model as compared to the base model. In CodeBleu, the coding was assessed in four aspects, namely N-Gram Matching, Weighted N-Gram Matching, Syntax Matching, and Data Flow Matching. There was an appreciable increase in all four aspects of the fine-tuned model, which implies that fine-tuning helps in generating syntactically and logically correct code in Python.

#### 5.4.7 Functional Evaluation of base and fine-tuned model

The efficiency of the fine-tuning process was assessed by conducting a functionality analysis, employing the HumanEval benchmark, which involves programming problems aimed at evaluating the ability of a model to generate code. Identical prompts were used for both the models, and their generated code was assessed based on the following aspects:

- **Correctness:** Whether the generated code produces expected output for given test cases.
- **Code Quality:** Readability, structure, and efficiency of the generated code.
- **Error Handling:** Ability to handle edge cases and exceptions.

Quantitative metrics such as pass@k scores were calculated.

#### 5.4.8 Functional Evaluation of Fine-Tuned and Code-LLaMA Models

The capability of effectively producing code by the fine-tuned llama-2-7B model was assessed compared to the other Code-LLaMA model using the HumanEval dataset that consists of programming tasks. Identical prompts were fed into both models and the solutions produced by them were evaluated based on the following factors:

- **Correctness:** Whether the generated code produces the expected or reference output for given test cases.
- **Code Quality:** Readability, structure, and efficiency of the generated code.
- **Error Handling:** Ability to handle edge cases and exceptions appropriately.

We computed quantitative metrics such as pass@k scores were calculated.

#### 5.5 Ollama Implementation for Local Inference

Once the model is selected from the fine-tuning, the prompt will be passed to the local inference engine running on the Ollama platform. The implementation involves using the Ollama framework as a lightweight run-time platform that will execute the model on commodity hardware. First, the fine-tuned model will be exported in the GGUF format. When inference is performed, the prompt will be executed via Ollama. The Ollama platform will load the model into memory and infer. As a result, the real-time Python code generation occurs without relying on cloud computing resources. The Generate Code Solution use case reflects the capabilities of the system. When a prompt is received, the model will infer using learned parameters and generate syntactically and contextually correct Python code. The generated output will be passed to the response module in which all the extra tokenization and incomplete code fragments will be stripped off. As can be seen from the implementation perspective, there are many benefits of using local inference. Local inference protects data privacy, ensures lower latency, and allows full control over model behavior and configuration. In summary, implementing Ollama will enable efficient, scalable, and privacy-preserving deployment of the model.

##### 5.5.1 Use Case Diagram

The use case diagram represented in 5-9 depicts the way in which the interaction between the user and system components related to the Python code generation happens. This figure highlights the primary functionalities of the system and how external actors use functionalities provided by the system. There are two main actors in this use case diagram, namely the User and Ollama Engine. The User is the primary actor who interacts with the system directly using prompts, and the Ollama engine represents the backend model for code generation. In the first use case, "Input Coding Problem," the user uses prompting

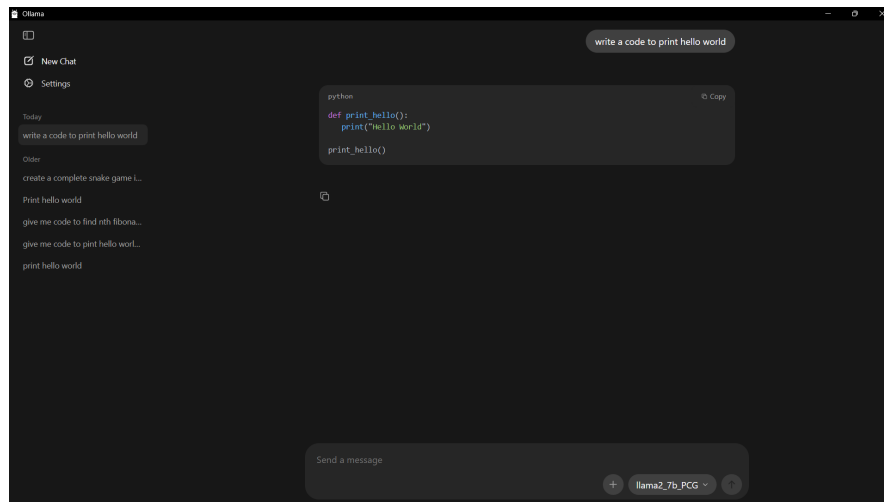


Figure 5-8 Ollama Inference

capability to input a Python programming problem. After entering the input, the user selects "Fine-Tuned Model use case" and chooses the model from a number of local models used in generating solutions to coding problems. "Generate Code Solution" is the most important function of this system, whereby the user's input and chosen model become its input and it generates the code. This use case is indicated with "includes" relationship since selecting a fine-tuned model is necessary for code generation. Also, there is an "Explain Algorithmic Logic" use case that provides explanations on the reasoning behind code generation solutions.

### 5.5.2 Class Diagram

The class diagram illustrated in Figure 5-10 depicts the static structure of the system. The User Interface class acts as the gateway for user interaction with the optimized system. It contains attributes such as `user_input` and `generate_code`, and provides methods like `getPrompt()` and `displayResult()` to get user input and display the generated output. The Prompt Formatter class formats the input prompt before it is sent to the model for code generation. It has attributes such as `template` and `example`, and methods like `formatPrompt()` and `add_example()` to structure the prompt in a way model accepts and add examples for few shot learning. The User Interface interacts with the Prompt Formatter in a one-to-many (0..n to 1) cardinality, indicating that multiple user inputs can be processed by a single Prompt Formatter instance. The Fine-tuned Model Manager class manages the loading of the trained models. It maintains attributes such as `base_model` and `adapter_model`, and provides the `loadModel()` method to inject the adapter layer to the base model. The Ollama Client class combines the application and the fine-tuned model, where prompt received from ollama UI is passed to the model. This class contains attributes like `base_model`, `temperature`, and `api_url`, along with functions like `sendPrompt()` and `getResponse()`, that allow communication between the model; the

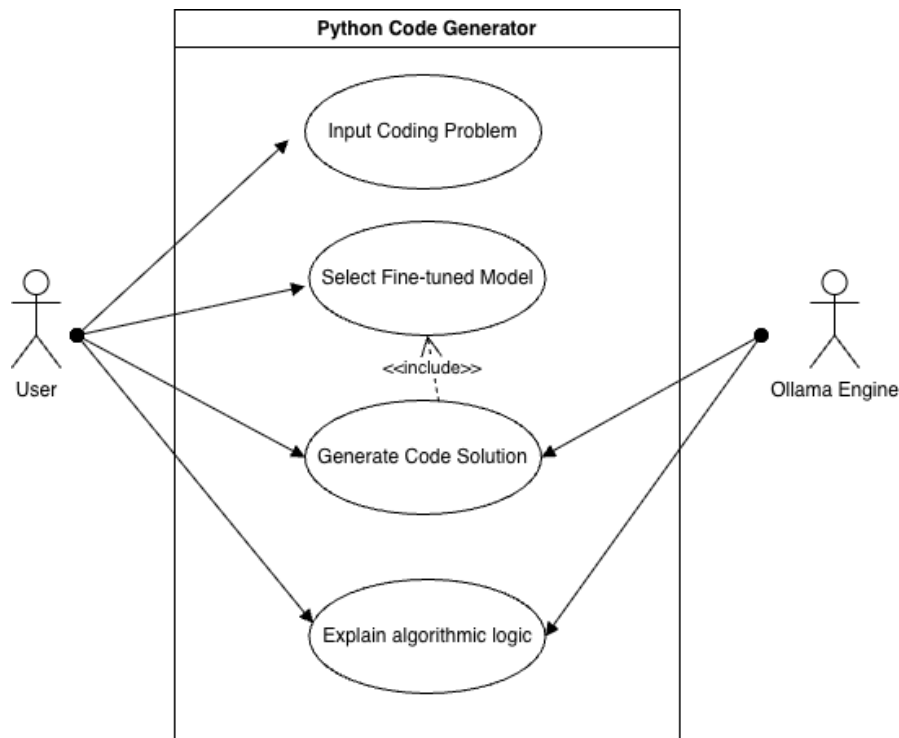


Figure 5-9 Use Case Diagram

function `sendPrompt()` sends the prompt passed from the user interface to the fine-tuned model, and the `getResponse()` function gets the response of the fine-tuned model and passes it on to the Model Response Parser class. Both the classes, the Prompt Formatter and the Fine-tuned Model Manager, depend on the Ollama Client Class in a one-to-one association. The final class in the diagram is the Model Response Parser class, which processes the output of the model generated using its own attributes like `raw_output` and `extracted_code` along with functions like `cleanOutput()` and `extractFunction()`. This relationship is established between the Ollama Client and this particular class in a manner such that the output is structured appropriately before being passed to the User Interface. In general terms, the class diagram provides an illustration of good design in terms of object communication, whereby each class plays its own defined role.

### 5.5.3 Sequence Diagram

Figure 5-11 depicts the sequential flow for the generation of code by interaction between various components of the system from user prompt entry to code generation displayed on the frontend interface. The code generation process begins when the User makes an entry into the system via the Frontend UI, providing a prompt related to the Python programming task to be solved. This information gets submitted by the frontend UI to the Backend API as a request. As mentioned above, the Backend API acts as a mediator from frontend UI to prompt processor, and forwards the prompt received to the Prompt Processor module. Once the pre-processing of the prompt is completed, the formatted



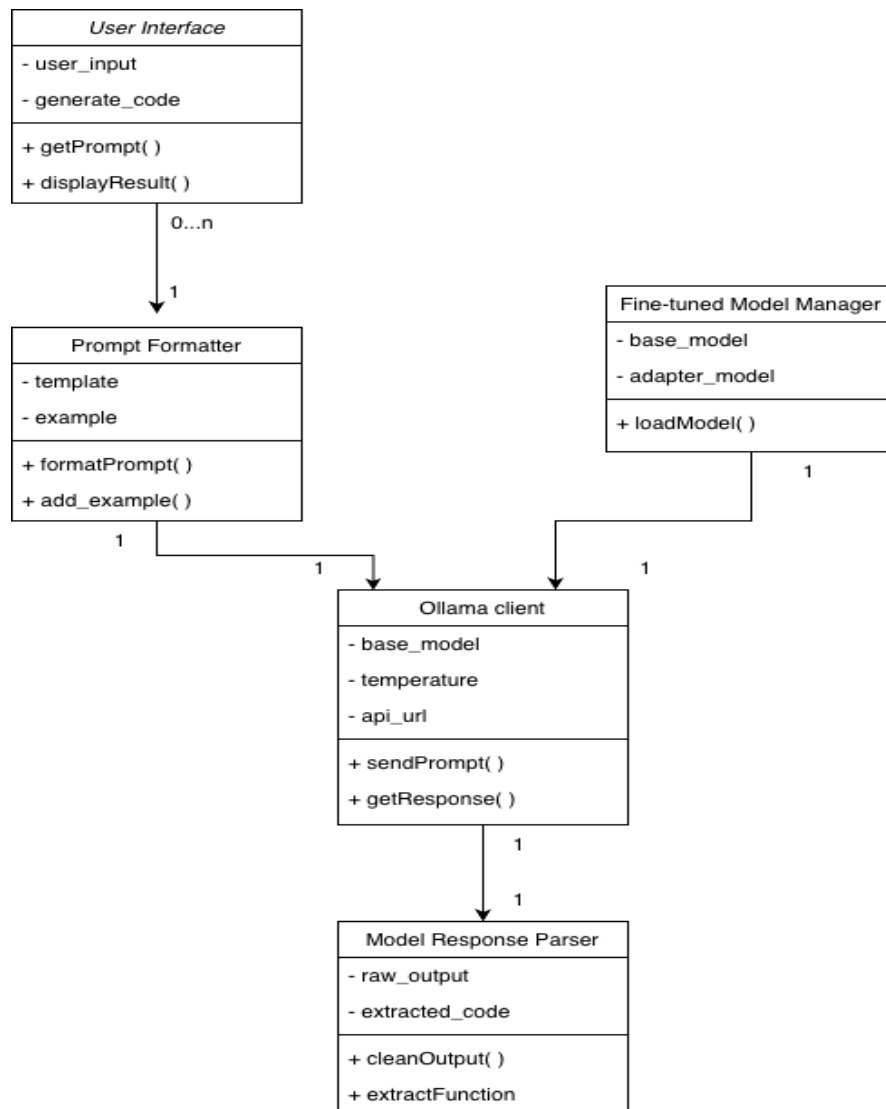


Figure 5-10 Class Diagram

prompt is delivered to the Ollama Client. This acts as the middleware module connecting the Backend API with the fine-tuned locally deployed model. The formatted request is then forwarded to the Fine-Tuned Model, wherein the actual computation happens. Based on this, the code output is then sent back through the same chain of modules in the reversed direction. Initially, the code output is sent by the model to the Ollama Client, and then the Client forwards it back to the Backend API, which sends it to the Frontend UI. Finally, the Frontend UI displays the output code to the User.

#### 5.5.4 Component Diagram

Figure 5-12 demonstrates the architecture of the system, showing the relationship between the frontend and backend components that contribute to the development of the python code response using the fine-tuned model. The system is divided into two major layers: the Frontend and the Backend. The frontend consists of the User Interface,

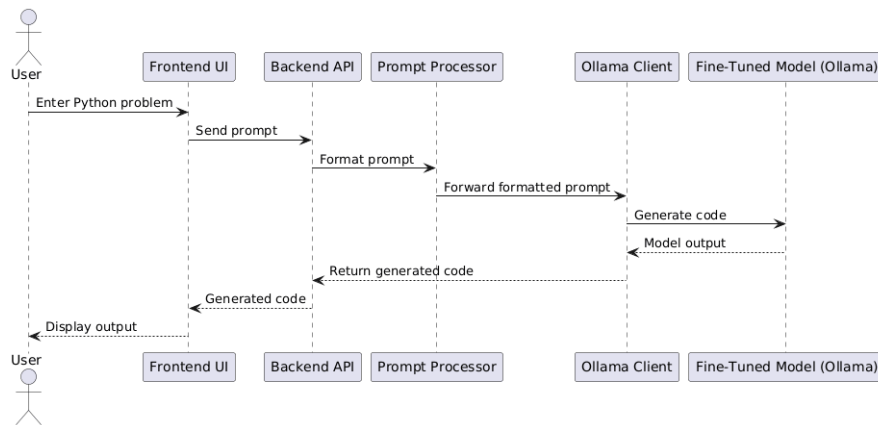


Figure 5-11 Sequence Diagram

Prompt Input Handler, and Result Viewer. The User Interface serves as the interaction point for users to input their prompts. These prompts are then processed by the Prompt Input Handler, which ensures that the input is properly formatted in a way model expects before being sent to the backend. On the backend side, the API Server receive requests from the frontend and forwarding them to the Prompt Processing Module. The Prompt Processing Module performs cleaning, formatting, and preparing the input prompt. The processed prompt is then forwarded to the Ollama Client, which then communicates with the locally hosted fine-tuned model. The Fine-Tuned Model (Ollama) generates the response based on the input prompt. Once the response is generated, it is sent to the Response Processing Module, which refines the raw response and formats the output for displaying in appropriate format. The final step will be sending the processed response to the Result Viewer component present in the frontend, where the response from the model will be shown to the user. The modularity of this approach allows for proper separation of concerns as well as easy communication between components that will carry out the code generation process.

### 5.5.5 Activity Diagram

Figure 5-13 is an activity diagram depicting the flow of operations within the system starting from user inputs to output production. It gives an overview of the flow of activities necessary for processing a query and producing a response to it. The process begins when the User enters a Python programming question through the frontend interface. The frontend then sends this prompt to the backend system for further processing. Upon receiving the request, the system performs prompt formatting to ensure that the input is structured correctly in the way model expects. Once the prompt is prepared, it is forwarded to the fine-tuned model for inference. The model processes the input and generates the corresponding Python code solution as output. After generating the code, the response undergoes a processing stage where it is refined and validated to

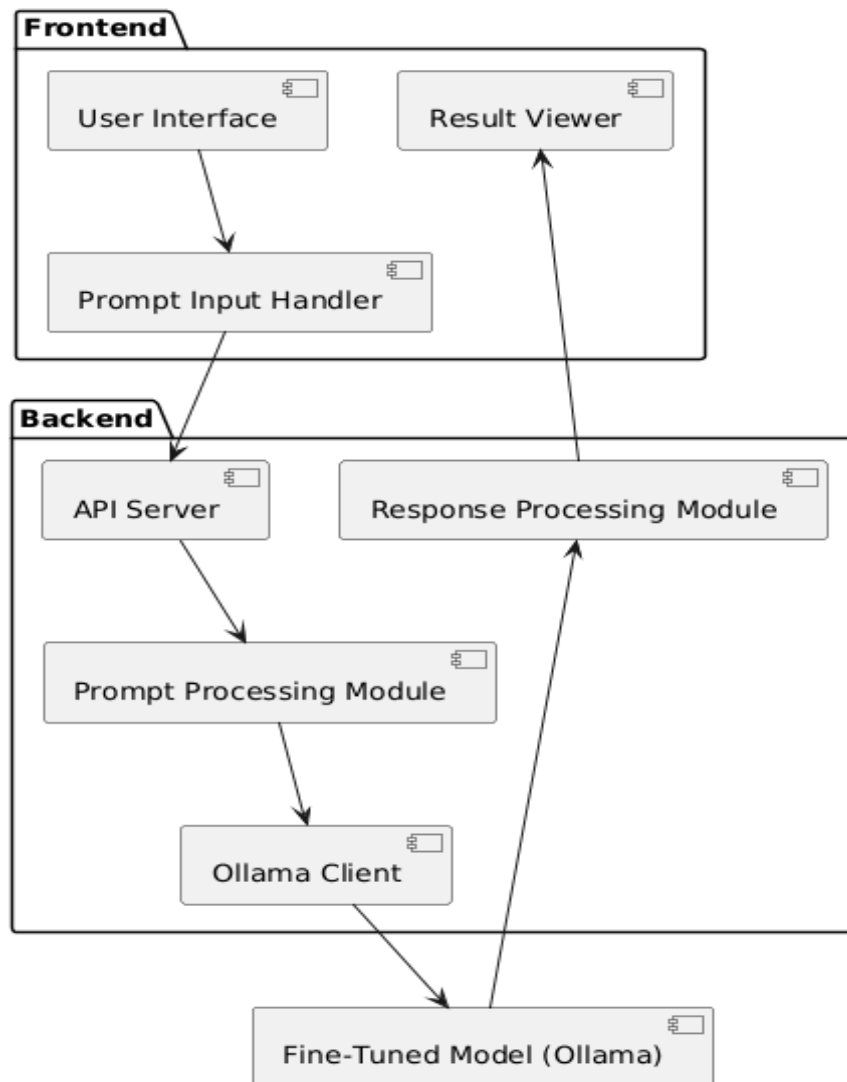


Figure 5-12 Component Diagram

check whether the generated code is valid. If the output is valid, the system proceeds to return the generated code to the frontend. However, if the output is not valid, an error message is displayed. This activity flow ensures that the system handles both successful and unsuccessful cases effectively.

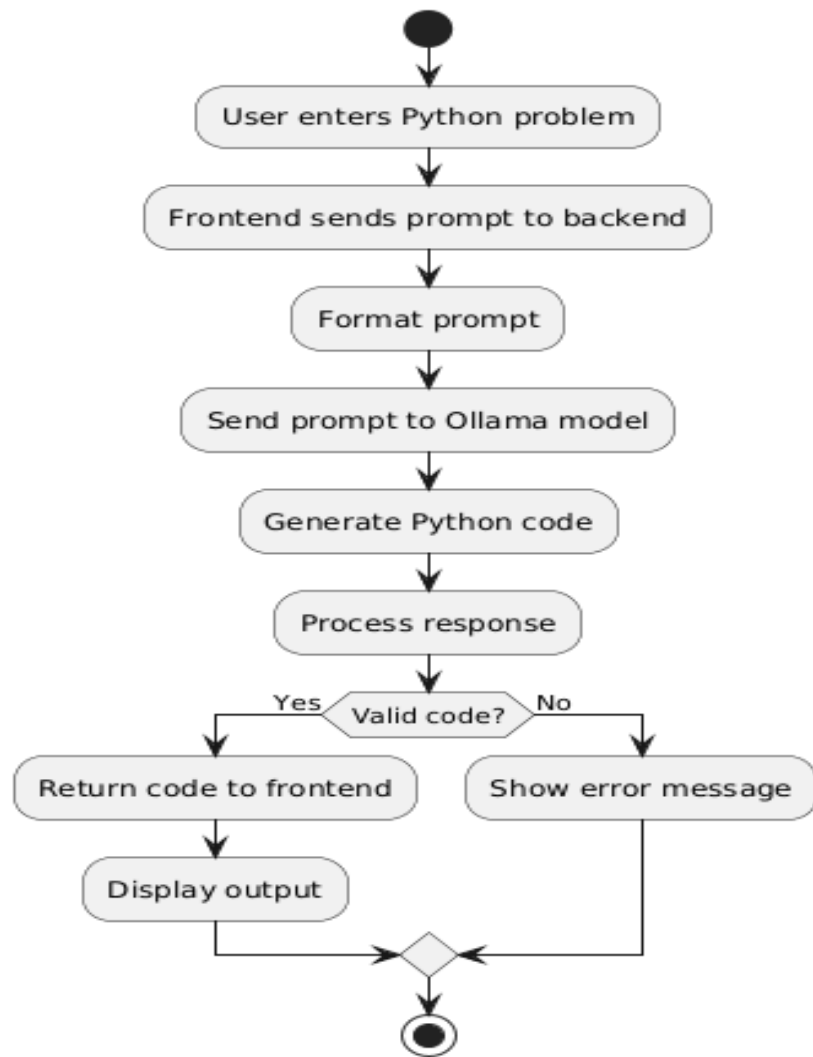


Figure 5-13 Activity Diagram

## 6 RESULTS AND ANALYSIS

### 6.1 Validation Loss During Fine Tuning

We evaluated the model using Cross-Entropy loss on a validation datasets in every 50 training steps. The losses for training and validation were decreasing in a steady way, implying the presence of a good learning curve without any overfitting.

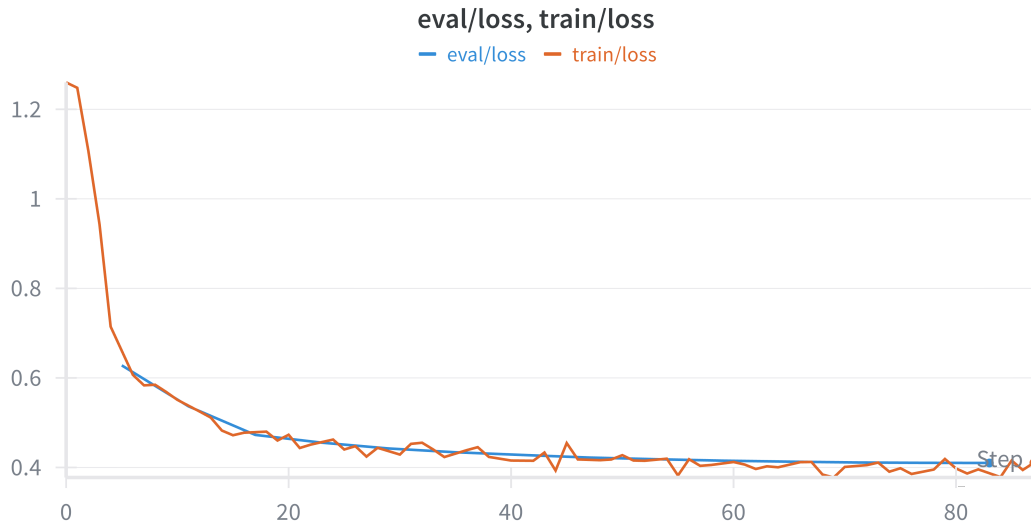


Figure 6-1 Training and Validation Loss Trajectory.

#### 6.1.1 Loss Convergence Analysis

It can be seen from Table 6-1 that during the first 150 steps, the model had its fast convergence of losses because the LoRA parameters adjusted to the syntax of Python at that time. Then, the validation loss plateaued at 600 step with a value of 0.41. The small difference between training loss (0.3957) and validation loss (0.4099) during 700 steps implies that the selected values of dropout probability (0.0961) and rank of LoRA  $r = 8$  are optimal.

Table 6-1 Detailed Training and Validation Loss Logs

| Step | Epoch  | Training Loss | Validation Loss |
|------|--------|---------------|-----------------|
| 50   | 0.1345 | 0.7145        | 0.6280          |
| 100  | 0.2690 | 0.5509        | 0.5364          |
| 150  | 0.4035 | 0.4776        | 0.4727          |
| 200  | 0.5380 | 0.4510        | 0.4553          |
| 250  | 0.6725 | 0.4438        | 0.4423          |
| 300  | 0.8070 | 0.4232        | 0.4339          |
| 350  | 0.9415 | 0.4154        | 0.4279          |
| 400  | 1.0753 | 0.4179        | 0.4221          |
| 450  | 1.2098 | 0.4148        | 0.4184          |
| 500  | 1.3443 | 0.4055        | 0.4151          |
| 550  | 1.4788 | 0.4003        | 0.4129          |
| 600  | 1.6133 | 0.4011        | 0.4111          |
| 650  | 1.7478 | 0.3854        | 0.4103          |
| 700  | 1.8823 | 0.3957        | 0.4099          |

## 6.2 Effect of Quantization and Performance Assessment

Quantization from BF16 to NF4 using QLoRA is an essential aspect of the approach. We conducted an analysis that looked at the balance between performance and model accuracy.

### 6.2.1 Memory and Computational Efficiency

The memory requirement of NF4 with double quantization was decreased from 10 GB(FP16) to 5.2 GB. The halving of VRAM helped us train the model on the Kaggle T4 GPU for free. In our experiments, we found that using NF4, a datatype optimized for normally distributed transformer weights, maintained the original capacity of the model.

### 6.2.2 Comparative Metric Analysis

To show the quantization effect, comparison of the performance between unquantized 16-bit and 4-bit quantized was done. We used CodeBLEU metric to account syntactic and structural integrity of code using Abstract Syntax Tree(AST) and dataflow matching.

Table 6-2 Performance Degradation Analysis: 16-bit vs. 4-bit Quantization

| Metric                  | 16-bit (Unquantized) | 4-bit (NF4)   | Delta ( $\Delta$ ) |
|-------------------------|----------------------|---------------|--------------------|
| SacreBLEU (Corpus)      | 5.8472               | 5.7232        | -0.1240            |
| <b>Overall CodeBLEU</b> | <b>0.2003</b>        | <b>0.1945</b> | <b>-0.0058</b>     |
| N-gram Match            | 0.0585               | 0.0572        | -0.0013            |
| Weighted N-gram         | 0.1050               | 0.1001        | -0.0049            |
| Syntax Match (AST)      | 0.2629               | 0.2537        | -0.0092            |
| Dataflow Match          | 0.3749               | 0.3670        | -0.0079            |

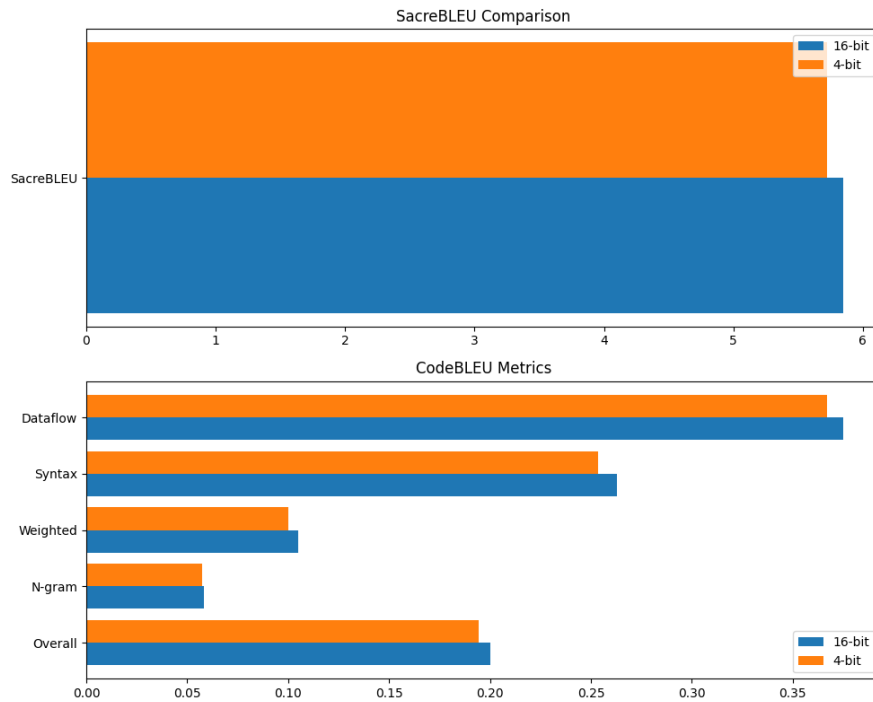


Figure 6-2 BLEU And CodeBLEU Score Of Quantized And Unquantized Model

### 6.2.3 Results Interpretation

The results in Table 6-2 show the effectiveness of the QLoRA approach:

1. **Minimal Loss of Structure:** We can observe that Syntax Match(AST) metric scores went down by 0.0092, meaning that the 4-bit model is maintaining almost an identical knowledge regarding the structural elements of Python language (such as loops, class, functions, definitions of variables).
2. **Maintaining Logic Flow:** High dataflow match score is maintained – 0.3670, meaning that variables are defined and used throughout the program accordingly to the correct sequence. The low score decrease confirms the fact that quantization process does not affect model capability which says that variables' states are preserved.

3. **Comparison between ScaREBLEU and CodeBLEU:** Although the ScaREBLEU scores appear low, this is expected for code generation as many variable names and whitespace variations can differ between generated and reference code. The stability shown by CodeBLEU indicates that dataflow and syntax matches are more reliable indicators of the model's performance in code generation.

#### 6.2.4 Conclusion on Quantization

The investigation on quantization reveals that NF4 quantization using 4 bits is a significant advancement in efficiency without compromising accuracy to a large extent. The difference in CodeBLEU (0.0058), representing only 2.8%, indicates that fine-tuning for quantization in local deployment is efficient with minimal impact on accuracy.

### 6.3 Hyperparameter Optimization and Training Stability

In order to achieve optimized training with the best parameter as such parameter may be able to reduce the loss quickly. For optimizing the hyperparameters, Optuna Bayesian optimizer was employed.

#### 6.3.1 Optimization Methodology and Search Space

Total 45 trials were conducted to reduce the validation cross-entropy loss. The desired hyperparameters governing the QLoRA adapter and training process are:

- **Learning Rate:** between  $1 \times 10^{-6}$  and  $5 \times 10^{-5}$ .
- **LoRA Rank ( $r$ ):** 8, 16, and 32.
- **LoRA Alpha ( $\alpha$ ):** 8, 16, and 32.
- **Scheduler Type:** Linear, Cosine, and Cosine with Restarts.
- **Batch Size:** 4, 8, and 16.

#### 6.3.2 Analysis of Trial Results

Hyperparameter optimisation process showed the strong correlation between lower batch size and improved coding performance on the training datasets.

1. **Early Convergence (Trials 0–9):** Initial trials focused on a LoRA rank of 8 and 16 with different Hyperparameter. The best initial value was achieved in trial 0 is (1.0179) with a linear scheduler.
2. **Scaling Breakthrough (Trials 11–20):** With the increase of LoRA rank to 32 and alpha parameter to 32, we viewed a significant performance jump. Trial 11 reduced the loss to 0.6384, which suggest that the higher rank allowed the adapters to capture more complex algorithmic patterns.



3. **Optimal Stability (Trials 40–44):** The final phase of optimisation showed that a cosine scheduler with a higher learning rate ( $2.9 \times 10^{-5}$ ) and low batch size (4) shows the most stable and lowest loss.

### 6.3.3 Final Hyperparameter Configuration

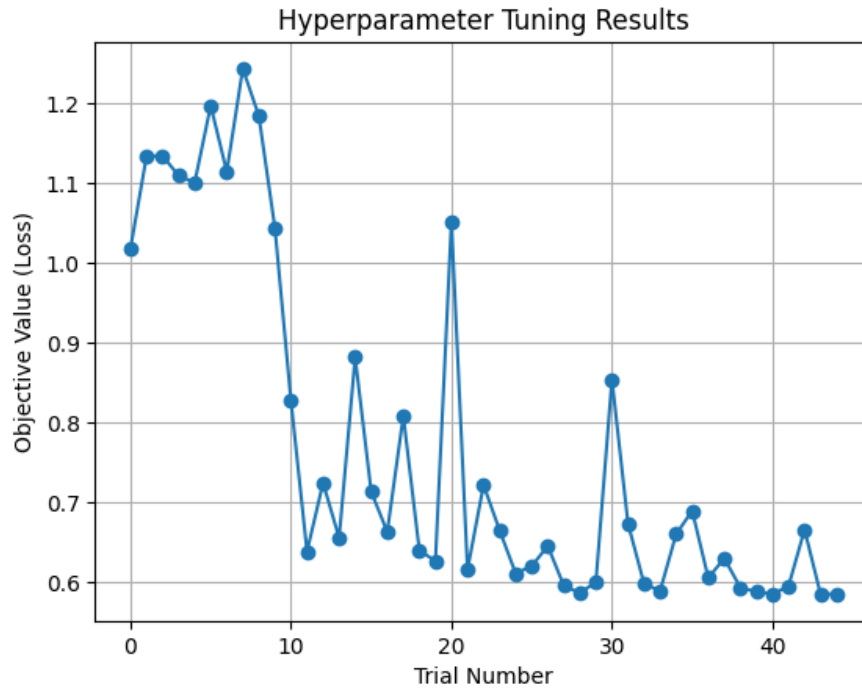


Figure 6-3 Evaluation Loss In Different Trials During Hyperparameter Tuning

The optimal parameters were found at trial 43 with a validation loss of 0.5847, which were selected for the final model production. Although a higher rank is often beneficial, a rank of 8 with high alpha 32 and a cosine scheduler provided the best configuration for efficient fine-tuning.

Table 6-3 Selected Optimal Hyperparameters (Trial 43)

| Hyperparameter          | Optimal Value           |
|-------------------------|-------------------------|
| Learning Rate           | $2.9089 \times 10^{-5}$ |
| LR Scheduler            | Cosine                  |
| LoRA Rank ( $r$ )       | 8                       |
| LoRA Alpha ( $\alpha$ ) | 32                      |
| LoRA Dropout            | 0.0961                  |
| Batch Size              | 4                       |
| Warmup Ratio            | 0.0693                  |

### 6.3.4 Discussion on Stability Training

From trial 0 to trial 43 where the loss was reduced from 1.0179 to 0.5847 there was an improvement of 42.5% in the stability training. Trials towards the 0.58 in the final stage (trial 40, 43, and 44) indicated success reaching the local plateau for the search space. The warm ratio of about 7% played a key role in stabilizing the weights of NF4 from gradient explosion.

## 6.4 Evaluation on Quantitative Metrics

Post-hyperparameter optimization phase, evaluation of the performance of the final fine-tuned Llama-2-7B was carried out by evaluating the model against a test dataset with 1,392 samples. The evaluation was done using the SacreBLEU and CodeBLEU metrics indicating the syntax similarity and function/structural alignment of solutions generated to the reference.

### 6.4.1 Evaluaiton Results

The results on 1,392 test samples are summarized in Table 6-4. The final model achieved a BLEU score of 10.27 and a CodeBLEU score of 0.2363.

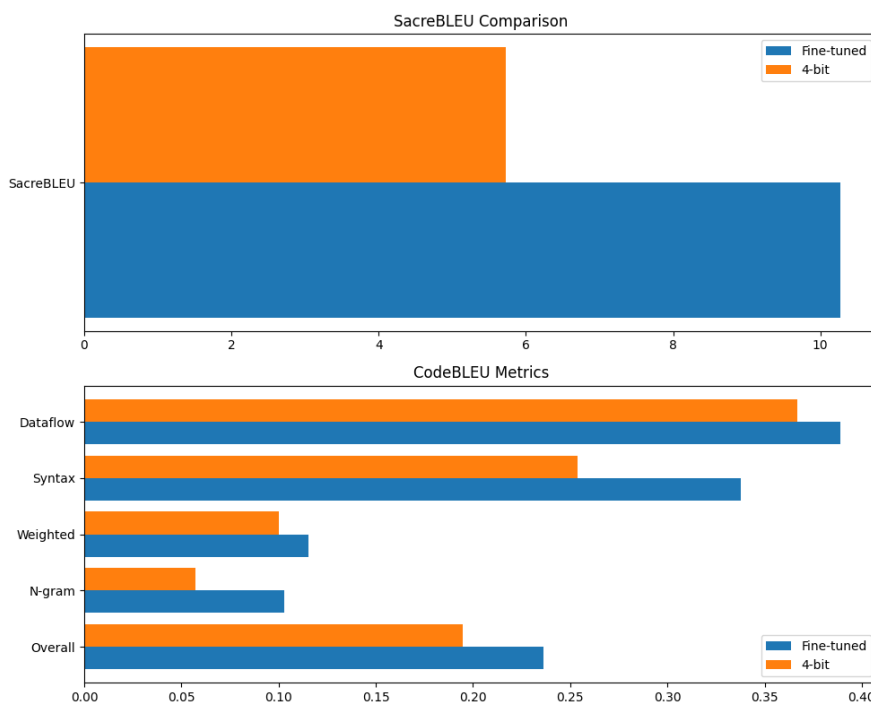


Figure 6-4 BLEU And CodeBLEU Score Of Fine-Tuned And Quantized Model

Table 6-4 Final Model Performance on Test Set ( $N = 1392$ )

| Metric Category          | Score         |
|--------------------------|---------------|
| Corpus BLEU (SacreBLEU)  | 10.2703       |
| <b>Overall CodeBLEU</b>  | <b>0.2363</b> |
| 1. N-gram Match          | 0.1027        |
| 2. Weighted N-gram Match | 0.1155        |
| 3. Syntax Match (AST)    | 0.3379        |
| 4. Dataflow Match        | 0.3889        |

#### 6.4.2 Technical Interpretation of Results

The CodeBLEU components scores revealed that there were some aspects as follows:

- **Structural Integrity:** The Syntax match (AST) score of 0.3379 indicated higher score compared to the n-gram score. From this result, one can conclude that the model is capable of creating Python codes with well defined structures regardless of differences in variable names.
- **Logical Consistency:** The dataflow match score of 0.3889 indicates strong performance. This proves that the model was able to maintain consistency in terms of using the right parameters in the function definition within the algorithm execution process.
- **BLEU Divergence:** The gap between the n-gram match and the weighted n-gram match (0.10 vs. 0.11), it is clear that the model has been taught to prioritize important functional tokens such as: 'def', 'return', 'if' and 'while'.

#### 6.4.3 Limitations of Evaluation Metrics

Although BLEU and CodeBLEU are useful, they have some inherent limitations when applied to code generation tasks.

- **BLEU Limitations:** BLEU takes the exact token matching into consideration and it cannot recognize the semantically similar programs which use different variable names and code structure.
- **CodeBLEU Limitations:** Though it considers the syntax and dataflow of the code, it doesn't always ensure its correct functioning as it could give correct syntax but incorrect function outputs.
- **Lack of Execution Semantics:** Since neither BLEU nor CodeBLEU runs the generated code and tests its output using other input data, they cannot provide sufficient testing of a code's correctness.

These types of problems resulted in the need for new metrics such as pass@k, which allows one to assess code functional correctness.

#### 6.4.4 Conclusion on Quantitative Evaluation

CodeBLEU score being 0.2363 in a large dataset containing 1,392 samples shows that the model has transitioned from a general language model to a specialized python code generator. The excellent result of the AST and Dataflow matching confirms that the model is capable of being used in a local code assistant, as these are the metrics which guarantee the code's successful execution.

### 6.5 Functional Evaluation: Pass@k on HumanEval

Though there are quantifiable metrics such as CodeBLEU indicating structural similarity between two models, the most reliable way to evaluate code generation ability is through functional evaluation. We have done an analysis of our model on humaneval datasets, which was developed by OpenAI and consists of 164 python programming problems. Each problem contains a functional signature, a docstring, and unit test cases to validate whether the answer provided by the model is accurate or not..

#### 6.5.1 Comparative Results

The performance of the base model was compared against the fine-tuned model over different values of k ranging from 1 to 5. The results are summarized in Table 6-5, and the results are consistent, indicating better performance for the model after fine-tuning..

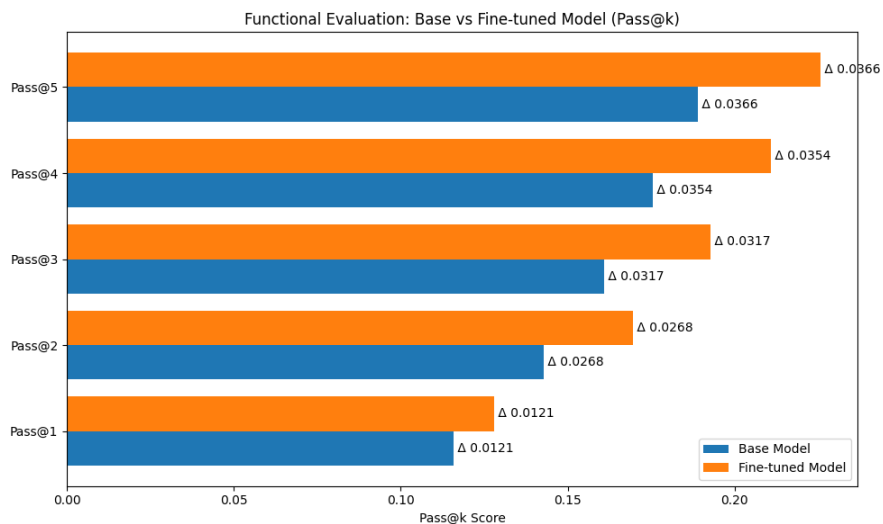


Figure 6-5 Pass@k Score Of Quantized And Fine-Tuned Model

Table 6-5 HumanEval Functional Correctness Results (Pass@k)

| <b>Metric</b> | <b>Base Model</b> | <b>Fine-Tuned Model</b> | <b>Improvement (%)</b> |
|---------------|-------------------|-------------------------|------------------------|
| Pass@1        | 0.1159            | <b>0.1280</b>           | +10.4%                 |
| Pass@2        | 0.1427            | <b>0.1695</b>           | +18.7%                 |
| Pass@3        | 0.1610            | <b>0.1927</b>           | +19.6%                 |
| Pass@4        | 0.1756            | <b>0.2110</b>           | +20.1%                 |
| Pass@5        | 0.1890            | <b>0.2256</b>           | +19.3%                 |

### 6.5.2 Performance Analysis

There are mainly three important conclusions drawn from the data:

1. **Baseline Growth:** When  $k = 1$ , the model performance increased significantly by 10.4%, showing that the performance of the model regarding code generation is accurate because of using high-quality data from the Flytech and Alpaca datasets.
2. **Enhanced Reasoning Ceiling:** It has been seen in the case of  $k=4$  and  $k=5$  where the percentage improvement has exceeded 20%. It indicates that while the fine-tuning of the base model increased the accuracy rate, it increased the model's diversity as well. It means the fine-tuned model will have the ability to follow the algorithmic path even when the base model is unable to find one.
3. **Consistency in Scaling:** As we have seen, the score difference of the base and fine-tuned models increases as 'k' increases. However, the base model's Pass@5 score (0.1890) is just slightly higher than the fine-tuned model's Pass@3 score (0.1927). In other words, fine-tuned 7B model requires less attempt to solve the same problem as compared to the larger unoptimized model.

### 6.5.3 Conclusion on Functional Testing

Outperformance of the fine-tuned model in all cases proves that tuning instructions to fit a particular domain is highly effective, even if we consider a smaller parameter footprint of 7B. With a Pass@5 score of 0.2256, it has been proved that for about a quarter of the problems included in the humaneval test, the fine-tuned model will deliver the functionally perfect output.

## 6.6 Functional Evaluation of CodeLlama

In order to prove a wider perspective of benchmarking, an assessment was performed on the performance of the CodeLlama model on the humaneval datasets through the Pass@k measure. This illustrates the performance of the specialized model compared to the fine-tuned model in terms of functionality.

### 6.6.1 HumanEval Results for CodeLlama

Table 6-6 illustrates the functional correctness scores of the CodeLlama model for various values of k. This table depicts the probability of having at least one correct solution among generated samples that pass all unit tests.

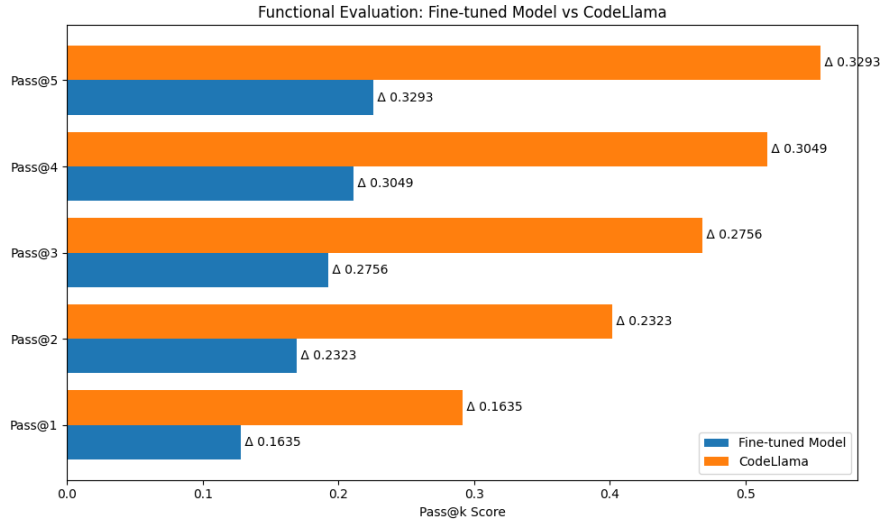


Figure 6-6 Pass@k Score Of Fine-Tuned And CodeLlama Model

Table 6-6 CodeLlama Functional Correctness Results (Pass@k)

| Metric | CodeLlama |
|--------|-----------|
| Pass@1 | 0.2915    |
| Pass@2 | 0.4018    |
| Pass@3 | 0.4683    |
| Pass@4 | 0.5159    |
| Pass@5 | 0.5549    |

### 6.6.2 Comparative Performance Analysis

Relative to our fine-tuned LLaMA-2-7B model, CodeLlama demonstrates the superior functional scores for all values of k. Specifically, CodeLlama achieves the Pass@1 functional score of 0.2915, which doubles our fine-tuned model in performance. The key strength of CodeLlama lies in its large-scale pretraining on code-specific corpora. The performance gain increases with k. For example, at k = 5, CodeLlama can achieve a functional score of 0.5549, meaning that it has the capacity to solve more than half of humaneval tasks in five tries. In contrast, our fine-tuned model can only achieve 0.2256 in the same task, which indicates their reasoning differences in capability and solution variety.

### 6.6.3 Discussion

The high performance of CodeLlama can be attributed to several factors:

1. **Scale and Pre-training Data:** CodeLlama is trained on a large code dataset to enable broader pattern recognition.
2. **Specialized Architecture:** Unlike general LLaMA-2 architectures, CodeLlama is specifically designed for code generation, including long context processing and token predictions.
3. **Higher Reasoning Diversity:** Higher functional scores as  $k$  increases imply that CodeLlama can find multiple pathways to solving problems.

### 6.6.4 Conclusion

Despite the improvements achieved with our fine-tuned LLaMA-2-7B model, it is evident from our experiments that its performance can hardly compete with that of CodeLlama because of the limitations imposed by parameter-efficient fine-tuning. On the other hand, it should also be noted that our model works under considerable constraints regarding the computational power (4-bit quantization and LoRA adaptation). The efficiency of this approach is justified considering the above-mentioned factors.

### 6.6.5 Error Analysis of Generated Code

The metrics such as Pass@ $k$  and CodeBLEU provide general information about the performance of the model. However, they cannot reveal the nature of the errors made by the model. It is crucial to examine the generated code to analyze the limitations of the fine-tuned model further.

#### Common Error Categories

The identified errors can be grouped into the following categories:

1. **Logical Errors:** The model generates syntactically valid code; however, there is an error in the logic implemented. For instance, employing a greedy method where a dynamic programming algorithm should be used.
2. **Edge Case Failures:** A majority of the solutions encounter edge cases more frequently compared to other cases like empty strings, single-item arrays, and high-value or negative numbers.
3. **Incorrect Loop Conditions:** The model occasionally outputs code where the loop does not terminate correctly.
4. **Variable Mismanagement:** Mistakes in variable initialization, reuse, and overwriting affect the program's data flow.

5. **Off-by-One Errors:** The off-by-one mistake occurs mostly in index-related tasks, particularly array traversals and slicing operations.

### Example Success Case

#### Problem:

Create a function `filter_by_substring(strings: List[str], substring: str) -> List[str]` that filters an input list of strings for only those containing the given substring.

#### Generated Output:

```
def filter_by_substring(strings: List[str], substring: str)
    -> List[str]:
    return [s for s in strings if substring in s]
```

### Insights from Success Case

The analysis of this example shows that the fine-tuned model exhibits:

- **Correct syntactic and structural Python code generation:** The function `filter_by_substring` is a valid and working piece of code that runs without any errors..
- **Correct logical implementation:** The function `filter_by_substring` is a valid implementation that can be executed successfully without any error.
- **Concise and readable code:** The solution is short due to using list in an efficient manner and can easily be understood by a human.
- **Robustness for typical inputs:** Can handle empty string, empty list and strings containing special characters.

These observations suggest that the model:

- **Effectively generalizes from training examples:** The model has the ability to translate natural language into python code.
- **Produces memory-efficient implementations:** The code is memory efficient due to avoiding unnecessary loops or creating additional variables.
- **Requires minimal human intervention:** The generated code needs no modification to run.

Future considerations for further fine-tuning include:



- Including edge cases, like very large lists and Unicode strings, for better performance under all conditions.
- Including optional parameters (e.g., case-insensitive filtering)..

### Example Failure Case

**Problem:** Create a function `def separate_paren_groups(paren_string: str) -> List[str]`. This string contains multiple groups of nested parentheses. Goal is to identify each independent, balanced group, separate them into individual strings, and return them as a list.

### Generated Output:

```
def separate\_paren\_groups(paren\_string: str) -> List[str]:
    res = []
    curr = ''
    prev = ''
    for char in paren_string:
        if char == '(':
            curr += char
            if prev == '(' and curr == '(' :
                res.append(curr)
                curr = ''
                prev = ''
            else:
                prev = curr
                curr = char
        elif char == ')':
            curr += char
            if prev == '(' and curr == '(' :
                res.append(curr)
                curr = ''
                prev = ''
            else:
                res.append(prev)
                prev = curr
                curr = char
        else:
            curr += char
    if prev != '':
```

```
        res.append(prev)
    return res
```

**Issue:** The generated code distinguishes between balanced parentheses and writes out proper python code. The code generated takes for granted the fact that the string passed to it is formatted properly and makes no checks if it's incorrectly formatted or contains any special characters. The logic appears to work on most cases but can be optimized even more.

### Insights from Error Analysis

The analysis of this example demonstrates that the fine-tuned model exhibits:

- Correct syntactic and structural Python code
- Accurate handling of parentheses
- Limitation on edge cases and malformed input
- Memory is not fully utilized for large inputs

These observations suggest that the model could benefit from:

- Fine-tuning on more diverse and structured samples of python code and response which can handle complex and edge-case inputs
- Reinforcement Learning from Human Feedback (RLHF) which improve the readable and logical reasoning of the model
- Preprocessing activities so that the model is reliable for more input scenarios

## 6.7 System Performance in Deployment (Ollama)

The final model was converted to GGUF format and deployed via Ollama. Performance on a local NVIDIA RTX 4050 showed an average inference speed of **15.67 tokens per second**, demonstrating that our fine-tuned 7B model is highly viable for local deployment environments.

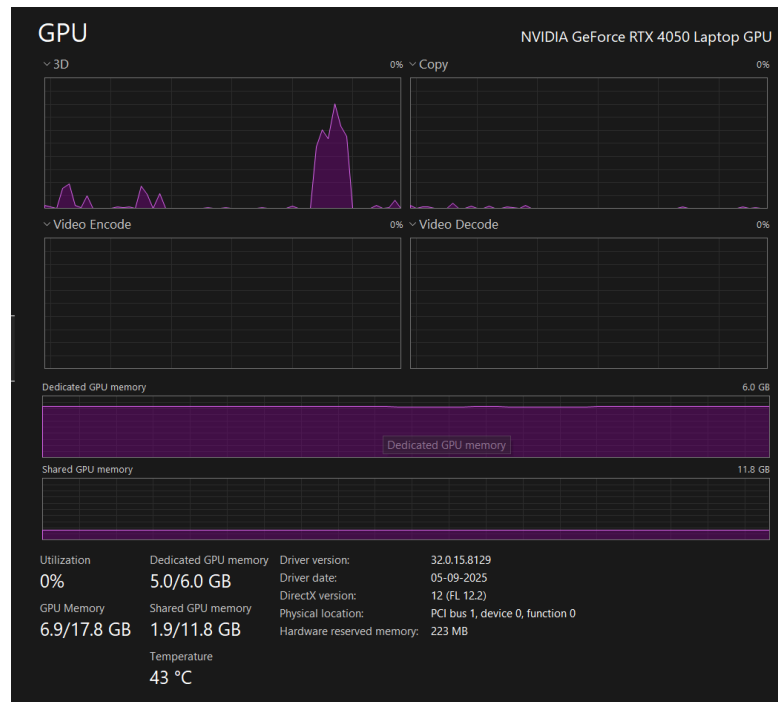


Figure 6-7 Resource Usage By Ollama During Inference

## 6.8 Summary

Combining QLoRA with high-quality, filtered datasets enables a 7B parameter model to achieve coding capabilities that previously required larger models. Local deployment via Ollama ensures minimal latency and maximum data privacy.

## **7 FUTURE ENHANCEMENT**

### **7.1 Limitations**

We have achieved intended objectives, but there are some limitations that were identified in the system.

1. The model is restricted to the 7B parameter due to the limitations of hardware. We have used QLoRA which have reduces the hardware needs but the larger model have higher potential to improve the model capabilities to generate the python code.
2. Datasets that we have fine-tuned have not more diversity which may have reduce the generalization capacity of the model.
3. The final performance gain was limited, fine-tuned model showed improvements but the improvements was not consistent on all evaluation settings.
4. The fine-tuning was done on python code so the final model is not able to generalized on other programming language without any new fine-tuning on those programming language's datasets.
5. Evaluation was heavily relied on BLEU and CodeBLEU score which is able to capture some semantic and dataflow score but it does not have ability to capture the logical or functional correctness of the generated code.
6. There is also a mismatch on the training datasets and evaluation datasets as the humaneval datasets contain difference type of input which contain the functional signature, parenthesis but the training datasets didn't contain this information .Training datasets were of only description as input.
7. Hyperparameter optimization was conducted for 45 trials but with the increase of number of trials ,we can obtain more better hyperparameter for fine-tuning.
8. The training datasets didn't contain the long context description and multi-file code so the final model was not capable to explored such type of capabilities.
9. The system behavior related to user interaction was not studied in the way that should have been done.
10. Reinforcement Learning from Human Feedback (RLHF) was not applied so the final model have the limitation of supervised fine-tuning.

### **7.2 Future Work**

The directions that needs to be pursued to make the system performance more capable:

#### **7.2.1 Reinforcement Learning Integration**

The current approach of fine-tuning was only relied on supervised method but the performance can be further improved through reinforcement learning, particular human

feedback can be used in reinforcement learning to optimize the model logical and functional capabilities as human intelligence is the one that we are trying to copy in our model.

In the context of python code generation, reinforcement learning is used make model more functionally correct, efficient and logical sound. We can set the reward function of reinforcement learning which satisfies the criteria such as code efficiency and execution correctness. This improves the response of the model by setting the expected rewards.

One of the potential method is to use the humaneval datasets and make a reward function as the generated code pass the test cases used. Code that passes most test cases is encouraged with higher reward policy. Adding the human feedback can even increase the readability and quality of the code.

Integration of reinforcement learning with existing supervised fine tuning would add a new phase of fine-tuning but this will lead to increase the generalisation capacity and performance on complex tasks.

Overall, reinforcement leaning presents a direction to enhance the capacity of the model for code generation, especially the capacity of handling the edge cases and producing optimized solutions.

### **7.2.2 Scaling Model Capacity**

We can fine-tune the larger model which have higher parameters in it as it have more reasoning depth and contextual understanding. It can help to determine the optimal trade off between the model size ,dataset size and hardware resources.

### **7.2.3 Dataset Expansion and Curriculum Learning**

We can explore new datasets which include the samples containing the programming scenarios to improve the performance of the model. This includes:

- Datasets containing different programming language
- Project level and real world scenario code
- Problems which are complex and competitive
- Sample which contain the debugging taks

Incorporating this Type of datasets the learning of the model can be improved to increase the model stability and performance.

### **7.2.4 Advanced Evaluation Strategies**

Future evaluation strategy should contains:

- Larger datasets for functional evaluation
- Security and vulnerability detection of generated code
- Human expert evaluation

These are some approach which can be used to show the comprehensive understanding of the model.

### **7.2.5 Retrieval-Augmented Code Generation**

We can integrate the retrieval-augmented generation to access the external knowledge and generate the solution to a certain coding problems. These external knowledge can be documentation of python code , API or any code repositories. This type of hybrid approach can improve the performance of the model to generate the code.

### **7.2.6 Deployment and System Integration**

Future enhancement can be focused for real world utilization of this model through:

- Integrating the model in an IDE for a certain project
- Optimize the model for real time scenarios
- Integrate the model into CI/CD pipelines

These type of improvements would make the system more practical for real world use and for real world applications.

## 8 CONCLUSION

The project have shown the development of a locally deployable AI-base code generation model using parameter efficient fine-tuning techniques. By using the LLaMA-2-7B model and optimization technique QLoRA we have developed a system with limited resource and use limited hardware resource for inference.

A structural pipeline was presented and implemented which contains phase like dataset preparation, instruction formatting, low-rank adaptation, and hyperparameter optimization. The use of QLoRA had enable the whole fine-tuning project in such low resource constraints.

Experimental results had shown that the fine-tuned model had achieved better code generation through BLEU, CodeBLEU, and functional correctness measure such as pass@k. These results shows the improvement in syntactic quality and semantic correctness of code generated by the model.

The system was also successfully deployed using Ollama which enable efficient local inference and we demonstrate that the model can be inference locally without using any cloud platform with only limited hardware resource found in consumer level gpu. This shows the improvement in accessible, privacy and cost efficiency.

### 8.1 Key Achievements

The major achievements of the project can be summarized as follows:

- Successfully implemented the QLoRA while fine-tuning for efficient adaptation of large language model.
- Presented a complete pipeline to fine-tune a model including data preprocessing and hyperparameter optimization.
- Showed the performance of final model through automated and functional evaluation metrics.
- Deployed the fine-tuned model using Ollama for local inference on the consumer level gpu hardware.

### 8.2 Final Remarks

The whole work in project highlights the growing importance of parameter-efficient techniques in developing a large language models which is more accessible and practical. This approach suggest that high-quality code generation can be achieved without any high computational power hardware resource.

With the improvement in dataset quality, evaluation strategies, and model scaling, these system can be improved and have a strong potential for real-world use case like software development and competitive programming. The project shows a solid foundation for future research in a code generation systems.



## 9 APPENDICES

### Appendix A: Mathematical Derivations

This chapter shows the details mathematical derivations of fine-tuning process, optimization algorithm and low-rank adaptation (LoRA) used in our project.

#### Low-Rank Adaptation (LoRA) Derivation

LoRA is used to reduce the parameters trained while fine-tuning with constraining the weight updates to a low-rank decomposition. For a pre-trained weight matrix  $W_0 \in \mathbb{R}^{d \times k}$ , the LoRA update is defined as:

$$\Delta W = BA$$

where  $B \in \mathbb{R}^{d \times r}$ ,  $A \in \mathbb{R}^{r \times k}$ , and  $r \ll \min(d, k)$  is the rank of the adaptation.

The forward pass for an input vector  $x \in \mathbb{R}^k$  becomes:

$$h = W_0x + \Delta Wx = W_0x + BAx$$

#### Gradient computation

While backpropagation, only matrix  $A$  and  $B$  are trained. Let  $\mathcal{L}$  denote the loss function. Chain rule is used to find the gradient of the loss with respect to LoRA parameters:

$$\frac{\partial \mathcal{L}}{\partial A} = B^\top \frac{\partial \mathcal{L}}{\partial h} x^\top$$

$$\frac{\partial \mathcal{L}}{\partial B} = \frac{\partial \mathcal{L}}{\partial h} (Ax)^\top$$

This formulae avoids the computing gradients for a large pre-trained weight, which reduces the memory usage and also computational cost.

## Appendix B: Hyperparameter Optimization Results

Table 9-1 Optuna Hyperparameter Optimization Results

| Trial | Learning Rate | Scheduler | LoRA r | LoRA $\alpha$ | Dropout | Batch Size | Loss   |
|-------|---------------|-----------|--------|---------------|---------|------------|--------|
| 0     | 1.31e-05      | linear    | 8      | 8             | 0.052   | 4          | 1.0179 |
| 1     | 1.02e-05      | cosine    | 16     | 32            | 0.129   | 16         | 1.1336 |
| 2     | 1.79e-05      | cosine_wr | 8      | 16            | 0.062   | 16         | 1.1340 |
| 3     | 1.92e-05      | cosine_wr | 16     | 8             | 0.142   | 8          | 1.1094 |
| 4     | 1.34e-05      | cosine    | 16     | 32            | 0.127   | 16         | 1.1006 |
| 5     | 2.10e-05      | linear    | 16     | 8             | 0.083   | 16         | 1.1972 |
| 6     | 9.37e-06      | cosine    | 8      | 8             | 0.053   | 4          | 1.1143 |
| 7     | 1.11e-05      | cosine    | 16     | 8             | 0.137   | 16         | 1.2436 |
| 8     | 2.33e-05      | linear    | 32     | 8             | 0.092   | 16         | 1.1840 |
| 9     | 1.23e-05      | cosine    | 8      | 8             | 0.054   | 4          | 1.0439 |
| 10    | 1.93e-05      | linear    | 8      | 8             | 0.063   | 4          | 0.8273 |
| 11    | 1.61e-05      | linear    | 32     | 32            | 0.073   | 4          | 0.6384 |
| 12    | 2.10e-05      | linear    | 32     | 32            | 0.084   | 8          | 0.7245 |
| 13    | 2.88e-05      | linear    | 32     | 32            | 0.061   | 8          | 0.6554 |
| 14    | 2.76e-05      | linear    | 32     | 32            | 0.056   | 16         | 0.8816 |
| 15    | 1.61e-05      | linear    | 32     | 16            | 0.105   | 4          | 0.7133 |
| 16    | 2.74e-05      | cosine    | 8      | 32            | 0.054   | 8          | 0.6630 |
| 17    | 2.84e-05      | cosine_wr | 32     | 16            | 0.077   | 8          | 0.8083 |
| 18    | 1.59e-05      | cosine_wr | 32     | 32            | 0.078   | 4          | 0.6399 |
| 19    | 1.79e-05      | cosine_wr | 32     | 32            | 0.105   | 4          | 0.6265 |
| 20    | 1.69e-05      | cosine_wr | 32     | 32            | 0.131   | 16         | 1.0515 |
| 21    | 1.98e-05      | cosine_wr | 32     | 32            | 0.095   | 4          | 0.6158 |
| 22    | 2.39e-05      | cosine_wr | 32     | 8             | 0.097   | 4          | 0.7218 |
| 23    | 1.32e-05      | linear    | 32     | 32            | 0.054   | 4          | 0.6650 |
| 24    | 2.08e-05      | cosine_wr | 32     | 32            | 0.116   | 4          | 0.6111 |
| 25    | 1.88e-05      | cosine_wr | 8      | 32            | 0.110   | 4          | 0.6201 |
| 26    | 1.53e-05      | cosine_wr | 8      | 32            | 0.130   | 4          | 0.6453 |
| 27    | 2.51e-05      | cosine_wr | 8      | 32            | 0.109   | 4          | 0.5957 |
| 28    | 2.90e-05      | cosine    | 8      | 32            | 0.113   | 4          | 0.5864 |
| 29    | 2.36e-05      | cosine    | 16     | 32            | 0.110   | 4          | 0.6001 |
| 30    | 2.97e-05      | cosine    | 8      | 32            | 0.099   | 16         | 0.8539 |
| 31    | 2.86e-05      | cosine    | 8      | 8             | 0.125   | 4          | 0.6735 |
| 32    | 2.43e-05      | cosine    | 16     | 32            | 0.127   | 4          | 0.5980 |
| 33    | 2.73e-05      | cosine    | 16     | 32            | 0.149   | 4          | 0.5891 |
| 34    | 2.80e-05      | linear    | 16     | 32            | 0.136   | 8          | 0.6603 |
| 35    | 2.41e-05      | cosine    | 8      | 32            | 0.148   | 8          | 0.6895 |
| 36    | 2.23e-05      | cosine    | 16     | 32            | 0.146   | 4          | 0.6054 |
| 37    | 2.63e-05      | cosine_wr | 16     | 16            | 0.120   | 4          | 0.6298 |
| 38    | 2.58e-05      | linear    | 8      | 32            | 0.103   | 4          | 0.5933 |
| 39    | 2.77e-05      | linear    | 8      | 32            | 0.089   | 4          | 0.5879 |
| 40    | 2.87e-05      | linear    | 8      | 32            | 0.081   | 4          | 0.5851 |
| 41    | 2.52e-05      | linear    | 8      | 32            | 0.085   | 4          | 0.5948 |
| 42    | 2.97e-05      | linear    | 16     | 8             | 0.108   | 4          | 0.6658 |
| 43    | 2.91e-05      | cosine    | 8      | 32            | 0.096   | 4          | 0.5847 |
| 44    | 2.88e-05      | cosine    | 8      | 32            | 0.094   | 4          | 0.5853 |

## **Appendix C: GitHub Repository**

[https://github.com/sasudo2/Finetuning\\_llama2-7B\\_for\\_python\\_code\\_generation](https://github.com/sasudo2/Finetuning_llama2-7B_for_python_code_generation)

## Appendix D: Project Budget

The project budget is primarily allocated toward high-performance compute resources required for fine-tuning the 7-billion parameter model and the development of the local inference application.

### Software and Cloud Services

Since the project utilizes open-source models (Llama-2, Qwen-2.5) and open-source libraries (Transformers, PEFT, TRL), the primary cost is hardware acceleration.

Table 9-2 Estimated Cost Breakdown

| Category                    | Item/Description                                                                                                    | Estimated Cost       |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------|----------------------|
| Development                 | Python, Ollama, PyTorch, Hugging Face API (Open Source)                                                             | Free                 |
| Cloud Compute               | <b>Google Colab Pro / RunPod:</b><br>- NVIDIA A100 (40GB) for Fine-tuning<br>- NVIDIA L4 for dataset classification | \$150 - \$200        |
| Storage                     | Hugging Face Hub (Model Versioning) and Drive                                                                       | \$20                 |
| Miscellaneous               | Documentation, LaTeX Overleaf Premium (1 Month)                                                                     | \$15                 |
| <b>Total Estimated Cost</b> |                                                                                                                     | <b>\$185 - \$235</b> |

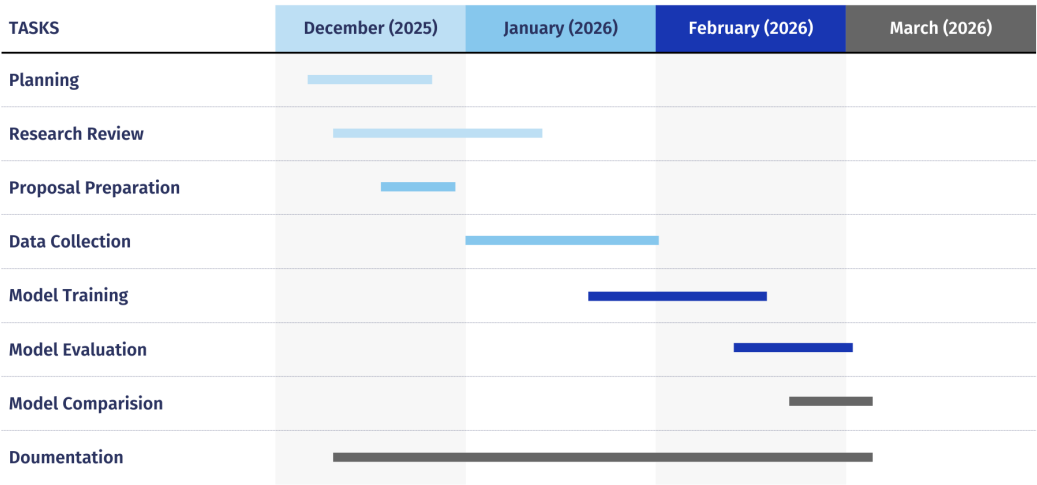
Appendix E: Project Timeline

The development followed a 6-month trajectory, from data acquisition to local deployment.

Gantt Chart

The following phases were executed:

Figure 9-1 Project Schedule (Gantt Chart)



## Appendix F: Consultation Form

**TRIBHUVAN UNIVERSITY**  
**INSTITUTE OF ENGINEERING, THAPATHALI CAMPUS**  
**Department of Electronics and Computer Engineering**  
**Student & Supervisor Consultation Form**  
**(BCT Minor Project, Batch 2079)**

|                                       |                                                                                                                                |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Project Title                         | Fine tuning Llama-2 7B for Python Code generation                                                                              |
| Group Members<br>(Name & Roll Number) | Pradip Pokhrel (THA079BCT027)<br>Deep Shrestha (THA079BCT011)<br>Rohan Dhakal (THA079BCT033)<br>Sanjay Shrestha (THA079BCT039) |
| Name of Supervisor                    | Asst. Prof. Suvarna Lingden                                                                                                    |

| S.N. | Brief Summary of Discussion Agenda                                                                                                                      | Date       | Supervisor Signature |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------|----------------------|
| 1    | Discussed on the key ideas of project deliverables. Asked to design the methodology of the project more clearly and advised to meet at 4pm 8/09/2021    | 2021/09/08 | <i>[Signature]</i>   |
| 2    | Checked the proposal draft and allowed the use of four different models, suggested the use of 11 more references and submit the draft by 3pm 16/09/2021 | 2021/09/09 | <i>[Signature]</i>   |
| 3    | Discussed the progress of the project; no. of epochs to use, using early stopping, figure out fine tuning of model; use of diagrams for representation. | 2021/10/7  | <i>[Signature]</i>   |
| 4    | Checked the results of first epoch and indicated the further progress of epochs using colab pro version                                                 | 2021/10/11 | <i>[Signature]</i>   |
| 5    | Final meeting with the students before mid-term defense; instructed for a thorough coverage of the contents.                                            | 2021/10/26 | <i>[Signature]</i>   |
| 6    | Meeting for discussion on planning of incorporation of feedback provided during mid-term defense                                                        | 2021/11/5  | <i>[Signature]</i>   |
| 7    | Checked the progress of additions on project; model training and evaluation                                                                             | 2021/11/12 | <i>[Signature]</i>   |
| 8    | Performance evaluation checked and decision on tweaking of datasets                                                                                     | 2021/11/26 | <i>[Signature]</i>   |
| 9    | Review of preliminary report; checked to see the corrections to be made                                                                                 | 2021/12/03 | <i>[Signature]</i>   |
| 10   | Final review of report and final addresses to the team                                                                                                  | 2021/12/16 | <i>[Signature]</i>   |

Date of Approval

Signature of Supervisor

At least **TWO** consultation is required before **Final Proposal Defense**  
 At least **FIVE** consultations are required **BEFORE Midterm** and **TEN** consultations for **FINAL**

## Appendix G: AI writing and Similarity Index

### AI writing



Page 2 of 106 - AI Writing Overview

Submission ID trn:oid::3117:592328097

#### \*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

#### Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

### Similarity Index



Page 2 of 115 - Integrity Overview

Submission ID trn:oid::3117:592328097

#### 10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

#### Filtered from the Report

- ▶ Bibliography
- ▶ Quoted Text
- ▶ Cited Text
- ▶ Small Matches (less than 8 words)
- ▶ Methods and Materials

## REFERENCES

- [1] J. Kaplan, S. McCandlish, T. Henighan, T. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020. [Online]. Available: <https://arxiv.org/abs/2001.08361>
- [2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023, meta AI. [Online]. Available: <https://arxiv.org/abs/2307.09288>
- [3] J. Howard and S. Ruder, “Universal language model fine-tuning for text classification,” *arXiv preprint arXiv:1801.06146*, 2018. [Online]. Available: <https://arxiv.org/abs/1801.06146>
- [4] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” *arXiv preprint arXiv:2305.14314*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.14314>
- [5] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021. [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [6] P. Diehl, N. Nader, M. Moraru, and S. R. Brandt, “LLM benchmarking with LLaMA2: Evaluating code development performance across multiple programming languages,” *arXiv preprint arXiv:2401.00000*, 2024.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [8] T. Brown, B. Mann, N. Ryder, D. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, G. Sastry, A. Askell, A. Herbert-Voss *et al.*, “Language models are few-shot learners,” *arXiv preprint arXiv:2009.10297*, 2020.
- [9] N. Houlsby, A. Giurghi, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for NLP,” in *Proceedings of the 36th International Conference on Machine Learning*, 2019.
- [10] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng, and M.-H. Chen, “DoRA: Weight-decomposed low-rank adaptation,” *arXiv preprint*



*arXiv:2402.09353*, 2024.

- [11] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, “OPT: Open pre-trained transformer language models,” *arXiv preprint arXiv:2205.01068*, 2022.
- [12] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. Canton Ferrer, A. Grattafiori, W. Xiong, A. Defossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, “Code llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2023, meta AI. [Online]. Available: <https://arxiv.org/abs/2308.12950>
- [13] S. Govande, T. Kang, and A. Shi, “Fine-tuning CodeLlama-7B on synthetic training data for Fortran code generation using PEFT,” *Stanford CS224N Custom Project*, 2024.
- [14] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: a method for automatic evaluation of machine translation,” in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002, pp. 311–318.
- [15] Q. Peng, Y. Chai, and X. Li, “HumanEval-XL: A multilingual code generation benchmark for cross-lingual natural language generalization,” *arXiv preprint arXiv:2402.16694*, 2024, IREC-COLING 2024. [Online]. Available: <https://arxiv.org/abs/2402.16694>
- [16] Z. Zhang and M. R. Sabuncu, “Generalized cross entropy loss for training deep neural networks with noisy labels,” in *Advances in Neural Information Processing Systems*, 2018.
- [17] B. Zhang and R. Sennrich, “Root mean square layer normalization,” *arXiv preprint arXiv:1910.07467*, 2019. [Online]. Available: <https://arxiv.org/abs/1910.07467>
- [18] N. Shazeer, “GLU variants improve transformer,” *arXiv preprint arXiv:2002.05202*, 2020. [Online]. Available: <https://arxiv.org/abs/2002.05202>
- [19] FlyTech, “Python codes 25k dataset,” <https://huggingface.co/datasets/flytech/python-codes-25k>, 2026, accessed: 2026-03-30.
- [20] iamtarun, “python\_code\_instructions\_18k\_alpaca dataset,” [https://huggingface.co/datasets/iamtarun/python\\_code\\_instructions\\_18k\\_alpaca](https://huggingface.co/datasets/iamtarun/python_code_instructions_18k_alpaca), 2026, accessed: 2026-03-30.

- [21] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma, “CodeBLEU: a method for automatic evaluation of code synthesis,” 2020, microsoft and collaborating universities.
- [22] OpenAI, “Humaneval: Hand-written python problems for code generation evaluation,” <https://github.com/openai/human-eval>, 2021, accessed: 2026-03-30.
- [23] Ollama, “Ollama,” <https://ollama.com>, 2026, accessed: 2026-03-30.
- [24] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyper-parameter optimization,” in *Advances in Neural Information Processing Systems 24 (NeurIPS 2011)*, 2011, pp. 2546–2554. [Online]. Available: <https://papers.nips.cc/paper/4443-algorithms-for-hyper-parameter-optimization>